TFHE from Scratch: A C++ Implementation of Gate Bootstrapping over the Torus

Matheus de Oliveira Saldanha

[1]Instituto de Computação – Universidade Estadual de Campinas (Unicamp)
Campinas – SP – Brazil

matheus.saldanha@dac.unicamp.br

Abstract. *Fully homomorphic encryption (FHE) lets a server compute on data that it cannot read. TFHE does this one Boolean gate at a time: after every gate, a procedure called bootstrapping cleans the noise of the result, so circuits of any depth can be evaluated. In this project we implemented TFHE gate bootstrapping from scratch in C++, using the same 128-bit parameters as the original TFHE library. This report explains the scheme step by step, from LWE encryption to a working encrypted adder, describes the main implementation choices (the negacyclic FFT and a trick that computes signed digits with a single addition), and compares our code with the original library. A NAND gate takes 22.9 ms in our implementation, which is* $2.3\times$ *slower than the original library with its assembly FFT and* $4.9\times$ *faster than its portable version. We saw no decryption errors in 12,500 gates, and the measured noise matches an average-case analysis.*

Resumo. *A criptografia totalmente homomórfica (FHE) permite que um servidor faça cálculos sobre dados que ele não consegue ler. O TFHE faz isso uma porta lógica de cada vez: depois de cada porta, um procedimento chamado* bootstrapping *limpa o ruído do resultado, e assim circuitos de qualquer profundidade podem ser avaliados. Neste projeto implementamos o* gate bootstrapping *do TFHE do zero em C++, com os mesmos parâmetros de 128 bits da biblioteca original. Este relatório explica o esquema passo a passo, da cifra LWE até um somador cifrado funcionando, descreve as principais escolhas de implementação (a FFT negacíclica e um truque que calcula dígitos com sinal com uma única soma) e compara o nosso código com a biblioteca original. Uma porta NAND leva 22.9 ms na nossa implementação,* 2.3 *vezes mais que a biblioteca original com a FFT em assembly e* 4.9 *vezes menos que a sua versão portável. Não observamos nenhum erro de decifração em 12,500 portas, e o ruído medido concorda com uma análise de caso médio.*

1. Introduction

The idea of computing on encrypted data was proposed by [Rivest et al. 1978], but the first scheme that supports any computation only came thirty years later, with the work of [Gentry 2009]. Today, practical fully homomorphic encryption (FHE) schemes are based on lattice problems, mainly Learning With Errors (LWE) [Regev 2005] and its version with polynomials [Lyubashevsky et al. 2010]. In these schemes every ciphertext contains

a small random error, called noise, which is needed for security. Each operation on ciphertexts makes the noise larger, and when it gets too large the message can no longer be decrypted. Gentry's solution was *bootstrapping*: an operation that takes a noisy ciphertext and produces a new ciphertext of the same message with small noise.

The first bootstrapping procedures took minutes. Building on the GSW scheme [Gentry et al. 2013], the FHEW scheme [Ducas and Micciancio 2015] reduced this to less than a second by bootstrapping after every Boolean gate. TFHE, proposed by [Chillotti et al. 2016a] and described in full by [Chillotti et al. 2020], improved this approach: it works over the real torus $\mathbb{T} = \mathbb{R}/\mathbb{Z}$ and introduces an *external product* between two kinds of ciphertexts. With it, a bootstrapped gate takes tens of milliseconds on one processor core, and the public key that the server needs shrinks from gigabytes to tens of megabytes. Because every gate gives back a ciphertext with fresh, small noise, any circuit can be evaluated, no matter how deep.

The goal of this project was to study TFHE, implement it from scratch in C++, and compare our implementation with the original library [Chillotti et al. 2016b] in four aspects: correctness, performance, memory and noise. Our implementation[1] has no external dependencies and uses exactly the same parameters as the original library. In this report we:

- explain the scheme step by step, showing for each part which problem it solves, with small examples (Sections 2 and 3);
- describe the implementation choices that matter most for correctness and speed (Section 4);
- compare our implementation with the original library and check that the measured noise agrees with the theory (Section 5).

Section 6 concludes.

How the scheme is presented. TFHE uses several kinds of ciphertexts and operations, and each one exists to solve a problem created by the others. Instead of defining all of them at the start, Sections 2 and 3 introduce them in the order in which the problems appear (Figure 1): first an encryption scheme with noise (LWE), then its two limitations, then the plan TFHE uses to fix both (looking up a value in a table by rotating it), then the two tools that this plan needs (TRLWE and the external product), and finally the five steps of a bootstrapped gate. Readers interested only in the implementation and in the results can go directly to Section 4.

2. Background: From Noisy Encryption to an Encrypted Table Lookup

This section builds, one step at a time, the tools that TFHE combines into a bootstrapped gate. Each subsection answers a question left open by the one before it. We follow the presentation of [Chillotti et al. 2020], in the setting used by the original library; the guide by [Joye 2022] explains the same material in more detail. We write $\mathbb{B} = \{0, 1\}$, $\lfloor x \rceil$ for the integer closest to x, and $\vartheta = \sigma^2$ for the variance of a random variable.

[1] The source code, the tests and the scripts that produce every number in this report are in the project repository (`github.com/zinho02/learning-tfhe`).

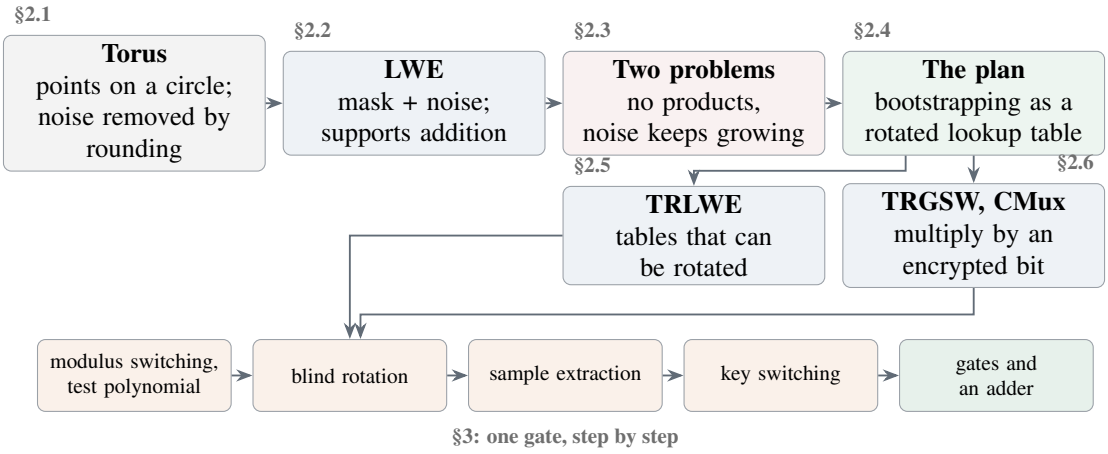


Figure 1. How the scheme is presented. Each concept of Section 2 answers a question left open by the one before it. The two tools of Sections 2.5 and 2.6 are what the plan of Section 2.4 needs, and Section 3 puts everything together in the five steps of a bootstrapped gate.

2.1. The Torus

All ciphertexts in this report work the same way: whoever has the key can compute the message plus a small error, and then removes the error by rounding. For this we need a set where messages can be added, where a small error can be removed by rounding to the nearest message, and where additions never overflow. The real torus $\mathbb{T} = \mathbb{R}/\mathbb{Z}$, the real numbers modulo 1, has all three properties. It is easiest to think of it as a circle of length one (Figure 2): adding a number moves us along the circle, and after a full turn we are back at the same point, so 0.25, 1.25 and -0.75 are the same element of $\mathbb{T}$.

Messages and rounding. A message $m \in \mathbb{Z}_M$ is placed at the point m/M, so the M possible messages are evenly spread around the circle. To decode a noisy point, we round it to the closest of these M points. This gives the right answer whenever the error is smaller than half the distance between two messages, that is, smaller than $1/(2M)$. For example, with $M = 8$ every point closer than $1/16$ to $1/8$ is decoded as $1/8$ (Figure 2a). The logic gates of Section 3.8 use an even simpler encoding: true is $+\frac{1}{8}$, false is $-\frac{1}{8}$, and to decode we only check in which half of the circle the point is, which allows errors up to $\frac{1}{8}$ (Figure 2b).

Integers times torus elements, but not torus times torus. We can multiply a point t of the torus by an integer k: $k \cdot t$ is just t added k times. But we cannot multiply two points of the torus, because the result would depend on which real number we use to represent each point. For example, $\frac{1}{2}$ and $\frac{3}{2}$ are the same point, but $\frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}$ while $\frac{3}{2} \cdot \frac{1}{2} = \frac{3}{4}$. This limitation has a big influence on the whole scheme: TFHE never multiplies two ciphertexts, and Section 2.6 explains how it still manages to multiply a ciphertext by an encrypted bit.

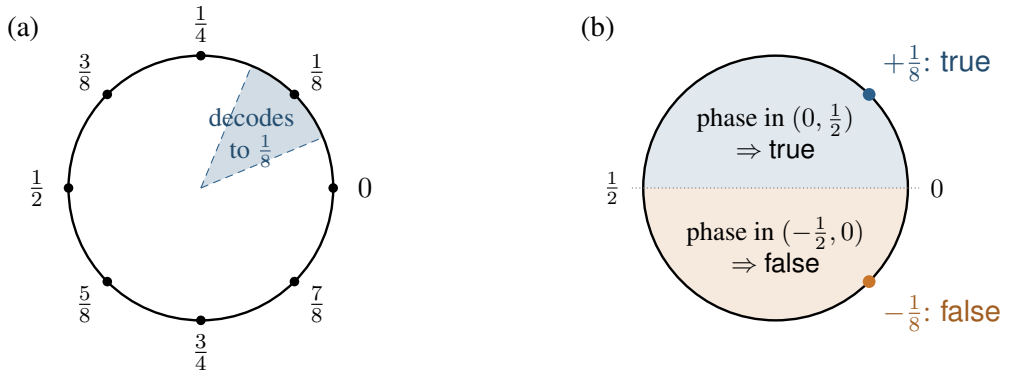


Figure 2. The torus $\mathbb{T} = \mathbb{R}/\mathbb{Z}$ drawn as a circle of length one. (a) The eight messages of $\mathbb{Z}_8$, placed at the multiples of $1/8$; the shaded slice contains the phases that decode to $1/8$, that is, all points at distance less than $1/16$. (b) The bit encoding used by the gates: true is $+1/8$ and false is $-1/8$. Decryption only checks in which half of the circle the phase is, so it allows an error of up to $1/8$ on either side.

2.2. LWE Encryption

A first attempt. Let the secret key be a list of n random bits, $s \in \mathbb{B}^n$. To hide a point $\mu \in \mathbb{T}$, we choose a random public vector $\mathbf{a} \in \mathbb{T}^n$ and publish the pair $(\mathbf{a}, b)$ with $b = \langle \mathbf{a}, s \rangle + \mu$. Only the owner of s can compute the *mask* $\langle \mathbf{a}, s \rangle = \sum_i a_i s_i$ and subtract it from b to recover μ. This scheme, however, is easy to break. Every ciphertext of a known message (for example, of 0) gives the attacker a linear equation in the n unknown bits of s, and with n such equations the attacker finds s by Gaussian elimination.

Adding noise. [Regev 2005] fixed this by making every equation slightly wrong.

Definition 1 (LWE ciphertext)**.** Let $s \in \mathbb{B}^n$ and $\sigma > 0$. An LWE encryption of $\mu \in \mathbb{T}$ is the pair $c = (\mathbf{a}, b) \in \mathbb{T}^{n+1}$, where $\mathbf{a}$ is uniform in $\mathbb{T}^n$, e is a random error drawn from a Gaussian with mean 0 and standard deviation σ, and $b = \langle \mathbf{a}, s \rangle + \mu + e$. The *phase* of c is $\varphi_s(c) = b - \langle \mathbf{a}, s \rangle = \mu + e$.

Because of the error e, the equations are only approximately true. Finding s from them, or even telling them apart from random numbers, is the Learning With Errors problem, which is believed to be hard even for quantum computers. So the noise is not a flaw of the scheme: it is what makes it secure. The cost is that, after removing the mask, we get the phase $\mu + e$ instead of μ. This is why messages are spread out on the circle and decoded by rounding (Section 2.1). The phase is the most important idea in this report: to know what a ciphertext contains, we compute its phase. For a new ciphertext with our parameters, $\sigma = 2^{-15} \approx 3 \cdot 10^{-5}$, which is more than two thousand times smaller than the tolerance $\frac{1}{16}$ of $\mathbb{Z}_8$.

Example 1 (A small LWE ciphertext)**.** To keep the numbers small, we use a torus with only 256 points (the integer t stands for $t/256$), $n = 4$, and messages in $\mathbb{Z}_4$, so that the message 1 is the point $1/4$, i.e., the integer 64. Take $s = (1, 0, 1, 1)$, $\mathbf{a} = (100, 37, 201, 54)$ and $e = 3$. Then $\langle \mathbf{a}, s \rangle = 100 + 201 + 54 = 355$ and $b = 355 + 64 + 3 \equiv 166 \pmod{256}$. To decrypt, we compute the phase $166 - 355 \equiv 67$ and round it to the closest multiple of 64, which is 64: the message is 1 and the error was 3.

2.3. What LWE Can and Cannot Do

Linear operations are free. The phase is a linear function of the ciphertext: for integers k_j, $\varphi_s(\sum_j k_j c_j) = \sum_j k_j \varphi_s(c_j)$. So if we add two ciphertexts component by component, we get a ciphertext of the sum of their messages. For example, if c_1 encrypts $\frac{3}{8}$ and c_2 encrypts $\frac{6}{8}$, then $c_1 + c_2$ decrypts to $\frac{9}{8} \equiv \frac{1}{8}$, which is addition modulo 8. Multiplying a ciphertext by a public integer works the same way. To add a public constant μ, we use the *trivial* ciphertext $(\mathbf{0}, \mu)$, whose phase is μ for any key. In short, anyone can compute linear functions of encrypted messages without knowing the key.

Two problems. Linear functions are not enough, and they also have a cost.

1. *No nonlinear operation.* To compute any function we need at least one nonlinear operation, such as the AND of two bits. LWE ciphertexts cannot be multiplied, since their components are points of the torus.
2. *The noise keeps growing.* When we add two independent ciphertexts, the variances of their errors add up, and multiplying by k multiplies the standard deviation by $|k|$. After enough operations, the error becomes larger than the tolerance and decryption fails.

Bootstrapping. [Gentry 2009] showed how to solve the second problem with *bootstrapping*: running the decryption itself on encrypted data, which replaces a noisy ciphertext with a fresh ciphertext of the same message. TFHE uses a version of bootstrapping that solves both problems at once. It takes an LWE ciphertext with phase φ, which may be quite noisy, and returns a new LWE ciphertext of $f(\varphi)$, where f is a function chosen in advance. The error of the result is small and does *not* depend on the error of the input. For logic gates, f tells in which half of the circle the phase is. This function is not linear, which solves the first problem, and its output has fresh noise, which solves the second.

2.4. The Idea of TFHE's Bootstrapping: a Table Lookup

How can a server compute $f(\varphi)$ if it only has ciphertexts and public keys and never learns φ? TFHE uses a table lookup, done by rotating a table, in three steps (Figure 3):

1. round the phase to one of a finite number of positions, $\bar{\varphi}$;
2. put in entry j of a table the answer for phases at position j, that is, $f(j/(2N))$, and *rotate* the table by $\bar{\varphi}$ positions, so that the answer for $\bar{\varphi}$ ends up at position 0;
3. read position 0.

The hard part is step 2: the server must rotate the table by an amount that it does not know, because $\bar{\varphi}$ depends on the secret key. To do this, we need two tools, which are described in the next two subsections:

(a) an encryption of a whole table that can be rotated cheaply and without increasing its noise: polynomials and TRLWE (Section 2.5);
(b) a way to make each rotation depend on a secret bit, that is, to compute "if the encrypted bit is 1 then rotate, otherwise do not": the external product and the CMux (Section 2.6).

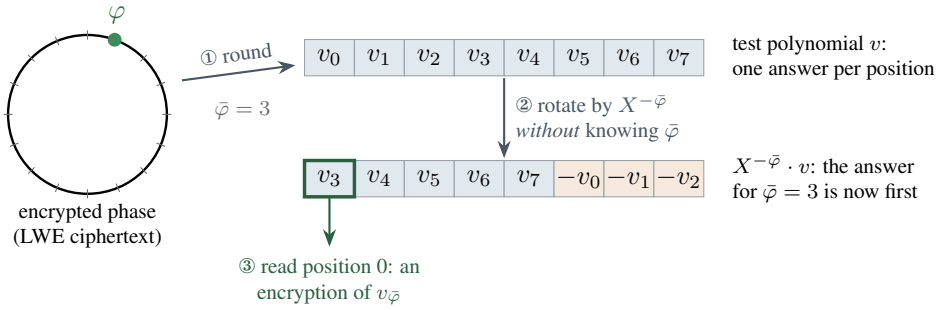


Figure 3. The idea of TFHE's bootstrapping, a table lookup by rotation, drawn with $N = 8$. ① The encrypted phase is rounded to one of $2N = 16$ positions. ② The table of answers, stored as the coefficients of a polynomial v, is rotated by that position, which brings the right answer to the front (the entries that wrap around come back with a minus sign). ③ The first entry is read. The hard part is step ②: the rotation depends on the secret key, so the server must rotate without knowing by how much.

2.5. Polynomials and TRLWE

Tables as polynomials. A table of N points of the torus can be written as a polynomial $v = v_0 + v_1X + \cdots + v_{N-1}X^{N-1}$, with one coefficient per entry. Then rotating the table should be the same as multiplying by a power of X. To keep the degree below N, we compute modulo $X^N + 1$: we work with $\mathbb{Z}_N[X] = \mathbb{Z}[X]/(X^N + 1)$ (integer coefficients) and $\mathbb{T}_N[X]$ (torus coefficients), where N is a power of two. The rule $X^N = -1$ turns multiplication by X into a rotation: every coefficient moves up one position, and the coefficient at the top comes back at the bottom with its sign changed. No arithmetic is needed, only moving coefficients and changing signs.

Example 2 (Rotation with $N = 4$)**.** $(1 + 2X + 3X^3) \cdot X^2 = X^2 + 2X^3 + 3X^5 = -3X + X^2 + 2X^3$, because $X^5 = X^4 \cdot X = -X$. With coefficient vectors, $[1, 2, 0, 3]$ becomes $[0, -3, 1, 2]$: every coefficient moves up two positions, and the one that falls off the end comes back at the start with its sign changed.

Because of the sign change, rotating N times changes the sign of the whole table ($X^N = -1$), and only after $2N$ rotations do we get back to the start ($X^{2N} = 1$). So the exponents of X work modulo $2N$, and a table of N entries has $2N$ rotation positions: in the first N positions we read the entries v_j, and in the next N we read $-v_j$. Section 3.3 shows that this sign change gives us exactly the sign function that the gates need. Products in $\mathbb{Z}_N[X] \times \mathbb{T}_N[X]$ are called *negacyclic*; Section 4.3 explains how we compute them quickly.

TRLWE: LWE with polynomials. If we replace every number in the definition of LWE by a polynomial, we get an encryption of a whole table at once.

Definition 2 (TRLWE ciphertext)**.** Let $s' \in \mathbb{B}_N[X]$ be a polynomial with coefficients in $\{0, 1\}$. A TRLWE encryption of $\mu \in \mathbb{T}_N[X]$ is a pair $c = (a, b) \in \mathbb{T}_N[X]^2$, where a is uniform, $e \in \mathbb{T}_N[X]$ has independent Gaussian coefficients, and $b = a \cdot s' + \mu + e$. Its phase is the polynomial $\varphi_{s'}(c) = b - a \cdot s' = \mu + e$.

Each coefficient of the phase is the matching coefficient of the message plus a small error. The phase is still linear, so TRLWE ciphertexts can be added just like LWE ci-

phertexts. What the plan needs is that they can also be rotated: if we multiply both components by X^e, we get a ciphertext with phase $\varphi_{s'}(X^e a, X^e b) = X^e b - X^e a\, s' = X^e \varphi_{s'}(c)$. Since X^e only moves coefficients and changes signs, the error is rotated together with the message but it does not grow. So rotating an encrypted table by a *known* amount is free. What is left is to rotate it by a secret amount.

2.6. Multiplying by an Encrypted Bit

The problem. The rotation in the plan depends on the phase $\varphi = b - \sum_i a_i s_i$, and therefore on the secret bits s_i. Section 3.4 will show that we can do it one bit at a time, with steps of the form

$$\mathrm{ACC} \leftarrow \begin{cases} X^{k_i} \cdot \mathrm{ACC} & \text{if } s_i = 1, \\ \mathrm{ACC} & \text{if } s_i = 0, \end{cases}$$

which is the same as

$$\mathrm{ACC} \leftarrow \mathrm{ACC} + s_i \cdot \left(X^{k_i} - 1\right) \cdot \mathrm{ACC},$$

where ACC is a TRLWE ciphertext, k_i is public, and the server only has s_i in encrypted form. So we need to multiply a TRLWE ciphertext by an encrypted bit, and there are two obstacles. First, the bit cannot be encrypted with LWE or TRLWE, because then we would have to multiply points of the torus by points of the torus. Second, even multiplying a ciphertext by a public integer is risky, because the error is multiplied too: the coefficients of a ciphertext are arbitrary points of the circle, and written as integers they are very large numbers. TFHE solves both problems in three steps.

Step 1: small digits. The second problem goes away if we never multiply by large numbers, only by small ones. For this, we write each point of the torus in a base B_g, using a few digits of small size.

Definition 3 (Gadget decomposition)**.** Let $B_g = 2^{\beta_{\text{bit}}}$ and $\ell \geq 1$. The decomposition of $t \in \mathbb{T}$ is a vector $\mathrm{Dec}(t) = (d_1, \ldots, d_\ell) \in \mathbb{Z}^\ell$ of signed digits $d_i \in [-B_g/2, B_g/2)$ such that

$$\left| t - \sum_{i=1}^{\ell} \frac{d_i}{B_g^i} \right| \leq \varepsilon := \frac{1}{2B_g^\ell}. \tag{1}$$

For a polynomial in $\mathbb{T}_N[X]$, we decompose each coefficient, and we get ℓ polynomials in $\mathbb{Z}_N[X]$ whose coefficients are at most $\beta = B_g/2$ in absolute value.

The vector $(B_g^{-1}, \ldots, B_g^{-\ell})$ that the digits multiply is called the *gadget*. With our parameters, $B_g = 2^7$ and $\ell = 3$, so each coefficient becomes three digits between -64 and 63, with a rounding error of at most 2^{-22}.

Example 3 (Signed digits in base 10)**.** The idea is easier to see in base 10 with $\ell = 2$. Then the digits must be in $[-5, 5)$ and $\varepsilon = 0.005$. For $t = 0.7316$, the usual digits are 7 and 3, but 7 is too large. Since $0.73 = 1 - 0.27$, on the torus 0.73 is the same point as -0.27, so the signed digits are $d_1 = -3$ and $d_2 = 3$. Indeed, $-\frac{3}{10} + \frac{3}{100} = -0.27 \equiv 0.73$, which is at distance $0.0016 \leq \varepsilon$ from t. Section 4.4 shows how to compute such digits quickly.

Step 2: the encryptions that the server needs (TRGSW). Let $d = (a, b)$ be a TRLWE ciphertext and m a secret integer (in our case, a key bit). We want a ciphertext whose phase is $m \cdot \varphi(d) = m \cdot b - m\,s' \cdot a$. If we decompose the two components into digit polynomials, $a \approx \sum_i u_i B_g^{-i}$ and $b \approx \sum_i w_i B_g^{-i}$, we get

$$m \cdot \varphi(d) \;\approx\; \sum_{i=1}^{\ell} w_i \cdot \left(\frac{m}{B_g^i}\right) + \sum_{i=1}^{\ell} u_i \cdot \left(\frac{-m\,s'}{B_g^i}\right). \tag{2}$$

The digits u_i and w_i are small and public, because the server can compute them from d. The terms in parentheses depend on the secret, so they must be given to the server in encrypted form. A TRGSW ciphertext of m is exactly the list of these 2ℓ encryptions.

Definition 4 (TRGSW ciphertext)**.** A TRGSW encryption of $m \in \mathbb{Z}$ under s' is a list of 2ℓ TRLWE encryptions of zero, $Z_1, \ldots, Z_{2\ell}$, to which we add m times the gadget: $C_i = Z_i + (m/B_g^i, 0)$ and $C_{\ell+i} = Z_{\ell+i} + (0, m/B_g^i)$ for $i = 1, \ldots, \ell$.

If we compute the phases of the rows, we see that they hold what Eq. (2) needs: $\varphi(C_i) = e_i - ms'/B_g^i$ (the term added to the first component gets multiplied by s' when we compute the phase) and $\varphi(C_{\ell+i}) = e_{\ell+i} + m/B_g^i$ (Figure 4). Since each row is an encryption, the TRGSW ciphertext does not reveal m.

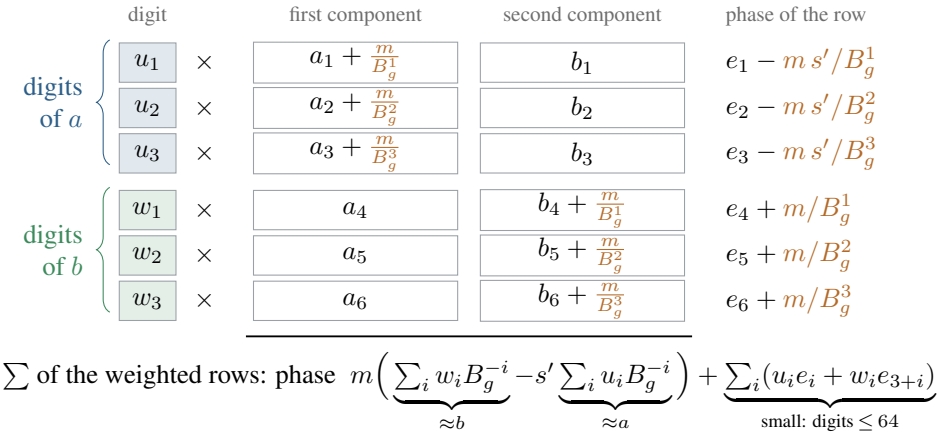


Figure 4. A TRGSW ciphertext of m, drawn as $2\ell = 6$ TRLWE rows (a_r, b_r) that encrypt zero, plus the gadget terms (orange). The external product multiplies each row by one digit polynomial, u_i for the digits of a and w_i for the digits of b, and adds the six rows. Adding the phases on the right, the gadget terms rebuild $m(b - a\,s')$ from the digits, while the noise is only multiplied by small digits.

Step 3: the external product. Now we just combine the rows using the digits as weights, which computes Eq. (2) on ciphertexts. The *external product* of C with $d = (a, b)$ is

$$C \boxdot d = \sum_{i=1}^{\ell} u_i \cdot C_i + \sum_{i=1}^{\ell} w_i \cdot C_{\ell+i}, \qquad (u_i) = \mathrm{Dec}(a),\ (w_i) = \mathrm{Dec}(b), \tag{3}$$

and, since the phase is linear,

$$\begin{aligned}\varphi(C \boxdot d) &= m\Big(\sum_i \frac{w_i}{B_g^i} - s'\sum_i \frac{u_i}{B_g^i}\Big) + \sum_i \big(u_i e_i + w_i e_{\ell+i}\big) \\ &= m\,\varphi(d) + m\,(\epsilon_b - s'\epsilon_a) + \sum_i \big(u_i e_i + w_i e_{\ell+i}\big),\end{aligned}$$

where ϵ_a and ϵ_b are the decomposition errors, whose coefficients are at most ε. Both problems are solved: we never multiply two points of the torus (the digits are integers), and the errors e_r of the rows are only multiplied by digits of size at most $\beta = 64$.

Proposition 1 (Noise of the external product [Chillotti et al. 2020])**.** *Let C encrypt $m \in \{0,1\}$ with noise variance ϑ_C, and let the error of d have variance ϑ_d. Then the error of $C \boxdot d$ has variance at most $2\ell N\beta^2\vartheta_C + m\big((1+N)\varepsilon^2 + \vartheta_d\big)$.*

The three terms come from the last equation. The first one comes from the 2ℓ products of a digit polynomial by an error polynomial: each coefficient of such a product is a sum of N terms, each a digit (at most β) times an error. The second comes from the decomposition errors, and the third is the error that d already had. The first term explains how the parameters are chosen. A larger base B_g needs fewer digits ℓ, so there are fewer products and the external product is faster, but β grows and so does the noise. It also explains why TRGSW ciphertexts need a very small error: with our parameters $2\ell N\beta^2 \approx 2.5 \cdot 10^7$, so they are created with standard deviation 2^{-25} instead of the 2^{-15} used for LWE (Table 4). Finally, the error of d is carried over but not multiplied, so the noise grows by *adding* a small amount each time. This is what allows hundreds of external products in a row.

The encrypted if-then-else (CMux). With multiplication by an encrypted bit, we can now choose between two ciphertexts without knowing the choice. Given TRLWE ciphertexts d_0 and d_1 and a TRGSW ciphertext C of a bit m,

$$\mathrm{CMux}(C, d_1, d_0) = d_0 + C \boxdot (d_1 - d_0) \tag{4}$$

encrypts $\mu_0 + m(\mu_1 - \mu_0)$, that is, the message of d_1 if $m = 1$ and the message of d_0 if $m = 0$, and the server does not learn which one (Figure 5a). It costs one external product in both cases. If we take $d_1 = X^{k_i} \cdot \mathrm{ACC}$, $d_0 = \mathrm{ACC}$ and C a TRGSW ciphertext of s_i, this is exactly the step that we needed at the start of this subsection.

Example 4 (CMux)**.** Let d_0 and d_1 encrypt the constant polynomials $\mu_0 = \frac{1}{8}$ and $\mu_1 = \frac{5}{8}$. The difference $d_1 - d_0$ encrypts $\frac{1}{2}$, so $C \boxdot (d_1 - d_0)$ encrypts 0 if $m = 0$ and $\frac{1}{2}$ if $m = 1$. Adding d_0 gives a phase of $\frac{1}{8}$ or $\frac{1}{8}+\frac{1}{2} = \frac{5}{8}$, which is the message of d_0 or of d_1 (Figure 5b).

3. How a Gate Is Bootstrapped

3.1. Overview

We now have everything we need to carry out the plan of Section 2.4 on real ciphertexts. Figure 6 shows the path of one gate. Each stage has a clear purpose:

- the *linear combination* of the two inputs gives a ciphertext whose phase is in the upper half of the circle exactly when the output of the gate should be true (Section 3.8);

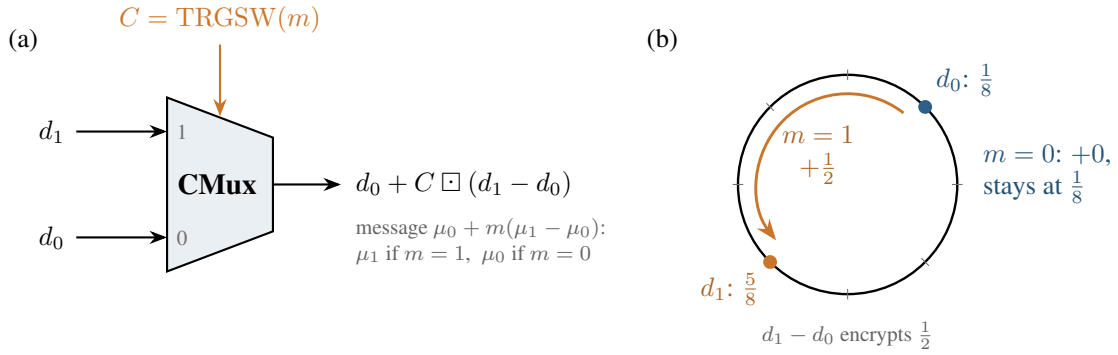


Figure 5. The CMux. (a) It chooses one of two TRLWE ciphertexts according to a bit m that is itself encrypted, using one external product. (b) The case of Example 4: d_0 encrypts $\frac{1}{8}$ and d_1 encrypts $\frac{5}{8}$, so $d_1 - d_0$ encrypts $\frac{1}{2}$. The external product adds $m \cdot \frac{1}{2}$ to the phase of d_0, which stays at $\frac{1}{8}$ when $m = 0$ and moves half a turn, to $\frac{5}{8}$, when $m = 1$. The server does the same work in both cases.

- *modulus switching* turns its phase, a point of the torus, into one of $2N$ rotation positions (Section 3.2);
- *blind rotation* rotates the *test polynomial*, which is the table of answers (Section 3.3), by that position, using one CMux per bit of the secret key (Section 3.4);
- *sample extraction* reads position 0 of the rotated table as an LWE ciphertext (Section 3.5);
- *key switching* brings this ciphertext back to the key and size of the inputs, so that it can be used by the next gate (Section 3.6).

Who has which key. The client has two secret keys: $s \in \mathbb{B}^n$, used to encrypt and decrypt the user's bits as LWE ciphertexts, and $s' \in \mathbb{B}_N[X]$, used for the TRLWE ciphertexts inside the bootstrapping. The server must run CMux operations controlled by the bits of s, so it receives each of these bits encrypted as a TRGSW ciphertext under s'. This is the *bootstrapping key*, $\mathrm{BK}_i = \mathrm{TRGSW}_{s'}(s_i)$ for $i = 1, \ldots, n$. The server also receives the *key-switching key*, described in Section 3.6. Together, they form the public *evaluation key*. Publishing it is considered safe under the usual assumption, called circular security, that each key can be encrypted under the other one.

3.2. Step 1: Modulus Switching

We can only rotate by whole numbers of positions, modulo $2N$ (Section 2.5), but the phase $\varphi = b - \sum_i a_i s_i$ of the input $c = (\mathbf{a}, b)$ is a point of the torus. So the first step cuts the circle into $2N$ slices and asks in which slice the phase is. The server cannot compute φ, so instead it rounds every public component of c:

$$\bar{b} = \lfloor 2Nb \rceil\,, \qquad \bar{a}_i = \lfloor 2Na_i \rceil\,, \qquad \bar{\varphi} = \bar{b} - \sum_i \bar{a}_i s_i \bmod 2N \approx 2N\varphi.$$

The integer $\bar{\varphi}$ is the number of the slice (Figure 8a shows the slices for $N = 4$). It is still secret, because it depends on s, but now it is a valid rotation amount. Rounding adds a small error: each rounded value is off by at most $1/(4N)$, with variance $1/(48N^2)$, and

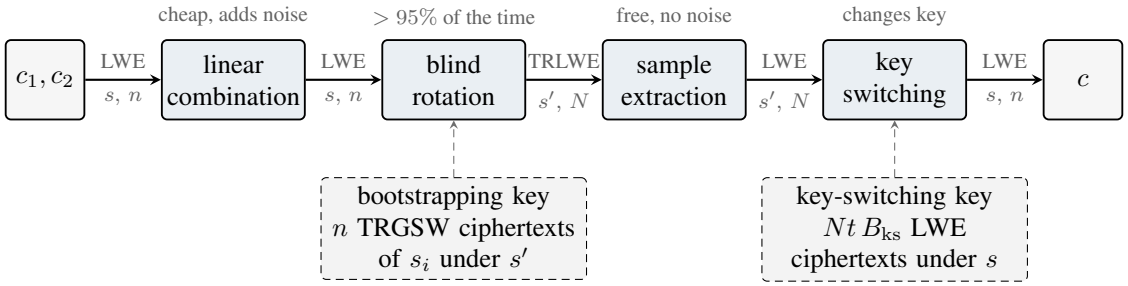


Figure 6. The path of a bootstrapped gate. The two input ciphertexts are combined linearly; blind rotation computes the gate on the encrypted phase and gives a TRLWE ciphertext with fresh noise; sample extraction reads its constant coefficient as an LWE ciphertext under the key s', of size N; and key switching brings it back to the key s, of size n, so that it can be used by the next gate. The dashed boxes are the two parts of the public evaluation key that the server has.

only $\bar{b}$ and the $\bar{a}_i$ with $s_i = 1$ (about half of them) matter. This *modulus-switching error* has variance $\vartheta_{\text{ms}} \approx (1 + n/2)/(48N^2) \approx 6.3 \cdot 10^{-6}$, a standard deviation of 0.0025. It adds to the error of the input and can at most make the lookup use a neighboring slice; it never reaches the output.

3.3. Step 2: The Test Polynomial

After modulus switching, the input has been reduced to a slice number $\bar{\varphi}$ between 0 and $2N - 1$. The bootstrapping must now return a value that depends on this slice, and the *test polynomial* is the table that says which value.

What the coefficients v_j are. The test polynomial $v = v_0 + v_1 X + \cdots + v_{N-1} X^{N-1}$ has torus coefficients chosen by the server. Each coefficient is one entry of the table:

> v_j is the value that the bootstrapping returns when the rounded phase of its input is $\bar{\varphi} = j$, that is, when the phase is close to $j/(2N)$.

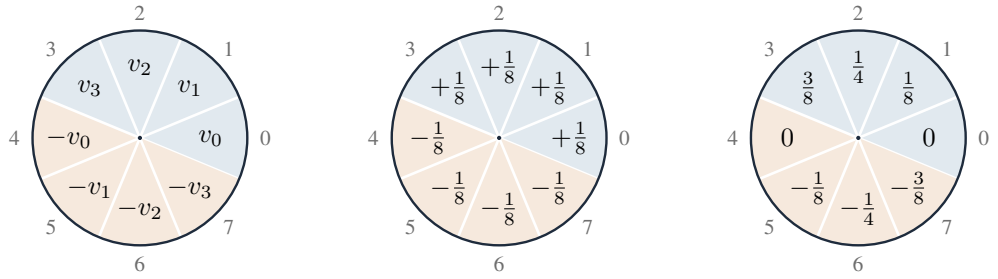
The N coefficients cover the N slices of the upper half of the circle. The slices of the lower half do not have their own coefficients: slice $N + j$, which is opposite slice j, returns $-v_j$. Figure 7 shows this for $N = 4$, with each of the 8 slices labeled with its output.

So, choosing v means choosing what the bootstrapping computes: to evaluate a function f of the phase, we set $v_j = f\big(j/(2N)\big)$ for $j = 0, \ldots, N - 1$. Figure 7 shows two examples.

- *Gates* (b): $v_j = \mu = \frac{1}{8}$ for every j. Every slice of the upper half returns $+\frac{1}{8}$ and every slice of the lower half returns $-\frac{1}{8}$. The bootstrapping computes the sign of the phase, already in the encoding of bits (true is $+\frac{1}{8}$, false is $-\frac{1}{8}$). The sign change caused by $X^N = -1$ gives us the second half of the sign function for free.
- *A table that is not constant* (c): with $N = 4$ and $v_j = j/8$, slice j of the upper half returns $j/8$. An input whose phase is close to $\frac{2}{8}$ comes out as an encryption of $\frac{2}{8}$, with fresh noise.

Since the lower half always returns the negated entries, the function must satisfy $f(\varphi + \frac{1}{2}) = -f(\varphi)$. The sign function does. Other functions can still be computed if we make

sure that the inputs are always in the upper half of the circle, as in the example at the end of this subsection.



(a) any test polynomial
$v = v_0 + v_1X + v_2X^2 + v_3X^3$

(b) gates: all $v_j = \frac{1}{8}$
$v = \frac{1}{8}(1 + X + X^2 + X^3)$

(c) $v_j = j/8$
$v = \frac{1}{8}X + \frac{1}{4}X^2 + \frac{3}{8}X^3$

Figure 7. What the coefficients v_j of the test polynomial mean, for $N = 4$. The circle is cut into $2N = 8$ slices, numbered by the rounded phase $\bar{\varphi}$ (outside), and each slice shows the value that the bootstrapping returns when the phase of its input is in that slice. Coefficient v_j is the output for slice j of the upper half (blue); the opposite slice $N + j$, in the lower half (orange), always returns $-v_j$, because $X^N = -1$. (a) The general rule. (b) The gate test polynomial: every v_j is $\frac{1}{8}$, so the output is $+\frac{1}{8}$ on the upper half and $-\frac{1}{8}$ on the lower half. (c) A table that is not constant, $v_j = j/8$: on the upper half the bootstrapping returns the rounded phase itself.

Why rotating reads the right entry. Now we can see why multiplying by $X^{-\bar{\varphi}}$ looks up entry $\bar{\varphi}$. Multiplying by X^{-1} moves every coefficient one position toward the constant term: the coefficient of X becomes the constant coefficient, and the old constant coefficient, which falls off the end, comes back as the coefficient of X^{N-1} with its sign changed. After $\bar{\varphi}$ such moves, the constant coefficient is $v_{\bar{\varphi}}$ if $\bar{\varphi} < N$, and $-v_{\bar{\varphi}-N}$ otherwise, which is exactly the label of slice $\bar{\varphi}$ in Figure 7. Figure 8b lists all $2N$ rotations for $N = 4$. Reading its first column from top to bottom gives $v_0, \ldots, v_3, -v_0, \ldots, -v_3$, the labels of the slices in order around the circle.

Is the test polynomial encrypted? No. The function being computed is not a secret (the server knows that it is computing a NAND, for example), so the table is public. What must stay hidden is the input, and therefore *which* entry of the table is selected. The test polynomial enters the blind rotation as the *trivial* TRLWE ciphertext $(0, X^{-\bar{b}} \cdot v)$. Its mask is zero, so its phase is $X^{-\bar{b}}v$ under any key, without any noise. This ciphertext hides nothing, but it does not need to, since $\bar{b}$ comes from the public part of the input. The accumulator becomes a real ciphertext at the first CMux, which adds $\mathrm{BK}_i \boxdot \big((X^{\bar{a}_i} - 1) \cdot \mathrm{ACC}\big)$. This term is a new encryption, under the real key s', of either the rotation difference (if $s_i = 1$) or zero (if $s_i = 0$). From then on the accumulator has a random mask and some noise, and the two possible results look the same to anyone without the key (Figure 9). In other words, the table is public, but the choice of the entry is secret.

Programmable bootstrapping. Using the bootstrapping to compute functions other than the sign was later called *programmable bootstrapping* [Chillotti et al. 2021]. For ex-

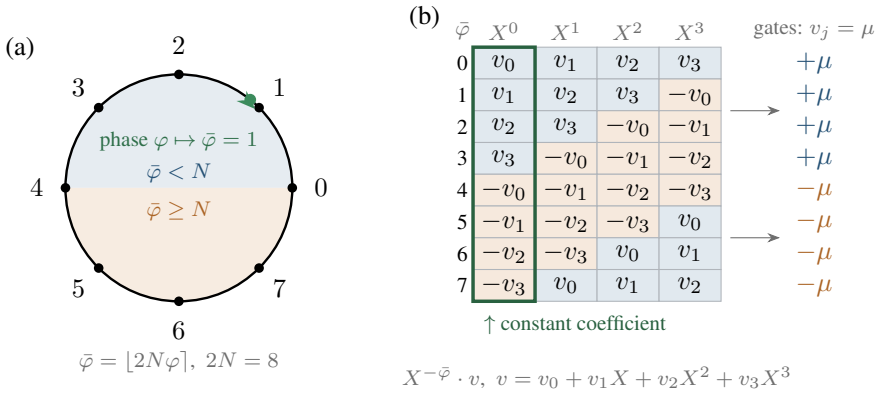


Figure 8. The test polynomial as a lookup table, drawn for $N = 4$. (a) The rounded phase $\bar{\varphi} = \lfloor 2N\varphi \rceil$ is one of $2N = 8$ positions on the torus; positions 0 to $N - 1$ are the upper half and N to $2N - 1$ the lower half. (b) The coefficients of $X^{-\bar{\varphi}} \cdot v$ for every $\bar{\varphi}$: each row is the row above shifted to the left, and the coefficient that leaves on the left comes back on the right with its sign changed, because $X^N = -1$. The constant coefficient (framed) is $v_{\bar{\varphi}}$ in the upper half and $-v_{\bar{\varphi}-N}$ in the lower half. With every $v_j = \mu$, the output is $+\mu$ or $-\mu$ depending on the half of the circle where the phase is: the sign function used by every gate.

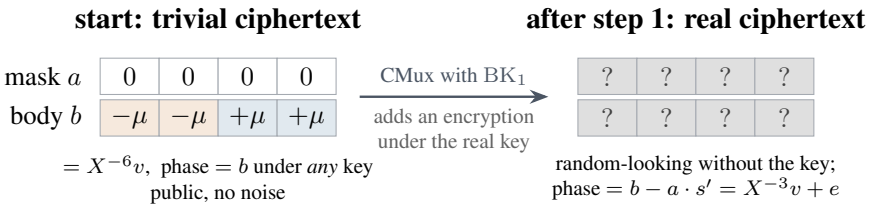


Figure 9. The test polynomial is not encrypted. Blind rotation starts from the trivial TRLWE ciphertext $(0, X^{-\bar{b}}v)$. Its mask is zero, so its phase is the public polynomial $X^{-\bar{b}}v$ under any key (here $N = 4$, $\bar{b} = 6$ and the gate test polynomial, as in Example 5). The first CMux adds an external product with a ciphertext of the bootstrapping key, which is a real encryption under s'. After it, the accumulator has a random mask and some noise, and without the key one cannot see whether it was rotated (here by X^3).

ample, with $N = 1024$ we can split the coefficients into four bands of $N/4 = 256$ consecutive coefficients and store $m/8$ in every coefficient of band m, that is, $v_j = \lfloor 4j/N \rfloor/8$. An input encrypting $(2m + 1)/16$ has a rounded phase close to $2N \cdot (2m + 1)/16 = 128\,(2m + 1)$, which is the middle of band m, so it comes out as an encryption of $m/8$. In this way one bootstrapping computes a lookup table with four entries instead of a sign. All these inputs are in the upper half of the circle, so the negated entries of the lower half are never used.

3.4. Step 3: Blind Rotation

We still need to compute an encryption of $X^{-\bar{\varphi}} \cdot v$ without knowing $\bar{\varphi}$. The main observation is that the rotation can be split into one factor per key bit:

$$X^{-\bar{\varphi}} = X^{-\bar{b}} \cdot \prod_{i=1}^{n} X^{\bar{a}_i s_i}, \qquad X^{\bar{a}_i s_i} = \begin{cases} X^{\bar{a}_i} & \text{if } s_i = 1, \\ 1 & \text{if } s_i = 0. \end{cases}$$

The first factor only uses public data. Each of the other factors is an if-then-else that depends on a secret bit, which is exactly what a CMux with $\mathrm{BK}_i = \mathrm{TRGSW}(s_i)$ computes. So the server starts from the trivial TRLWE ciphertext $\mathrm{ACC} = (0, X^{-\bar{b}}v)$ and, for $i = 1, \ldots, n$, replaces it by

$$\mathrm{CMux}\big(\mathrm{BK}_i,\ X^{\bar{a}_i} \cdot \mathrm{ACC},\ \mathrm{ACC}\big) = \mathrm{ACC} + \mathrm{BK}_i \boxdot \big((X^{\bar{a}_i} - 1) \cdot \mathrm{ACC}\big).$$

After the n steps, the accumulator encrypts $X^{-\bar{\varphi}} \cdot v$. The server knows every $\bar{a}_i$ and $\bar{b}$, but it does not know which of the n steps actually rotated, so it never learns the total rotation. This is why the operation is called *blind rotation*. Each step costs one external product and adds its error (Proposition 1), which is why the bootstrapping key must be so precise.

Example 5 (Bootstrapping an encryption of true, step by step)**.** Take $n = 3$, $N = 4$ (so there are $2N = 8$ positions), the secret key $s = (1, 0, 1)$, and the gate test polynomial $v = \mu\,(1 + X + X^2 + X^3)$ with $\mu = \frac{1}{8}$.

The input. Let c be an LWE encryption of true, that is, of $+\frac{1}{8}$, with error $e = 0.003$. With the mask $\mathbf{a} = (0.37,\ 0.61,\ 0.24)$, the body is $b = 0.37 \cdot 1 + 0.61 \cdot 0 + 0.24 \cdot 1 + 0.125 + 0.003 = 0.738$, and the phase is $b - (0.37 + 0.24) = 0.128$.

Step 1, modulus switching. We multiply by $2N = 8$ and round: $\bar{b} = \lfloor 5.904 \rceil = 6$ and $\bar{\mathbf{a}} = (\lfloor 2.96 \rceil, \lfloor 4.88 \rceil, \lfloor 1.92 \rceil) = (3, 5, 2)$. So $\bar{\varphi} = 6 - (3 \cdot 1 + 5 \cdot 0 + 2 \cdot 1) = 1$, a position in the upper half, as expected for a phase close to $\frac{1}{8}$. The server knows $\bar{b}$ and $\bar{\mathbf{a}}$, but not $\bar{\varphi}$.

Step 3, blind rotation. The accumulator starts as the trivial ciphertext $(0, X^{-6}v)$. Since $X^8 = 1$, we have $X^{-6} = X^2$, and $X^2 v = \mu(X^2 + X^3 + X^4 + X^5) = \mu(-1 - X + X^2 + X^3)$, because $X^4 = -1$ and $X^5 = -X$. The server then runs one CMux per key bit. It always does the same work, and the key decides which steps rotate:

step	$\bar{a}_i$	s_i	effect	message of ACC	coefficients
start	–	–	trivial ciphertext	$X^{-6}v$	$[-\mu, -\mu, +\mu, +\mu]$
$i = 1$	3	1	$\times X^3$	$X^{-3}v$	$[+\mu, -\mu, -\mu, -\mu]$
$i = 2$	5	0	unchanged	$X^{-3}v$	$[+\mu, -\mu, -\mu, -\mu]$
$i = 3$	2	1	$\times X^2$	$X^{-1}v$	$[+\mu, +\mu, +\mu, -\mu]$

From step $i = 1$ on, the accumulator is a real ciphertext (Figure 9). The table shows the *messages* it encrypts, which the server never sees.

Step 4, sample extraction. The constant coefficient of the final message is $+\mu = +\frac{1}{8}$, so the output is an encryption of true, the sign of the input phase, with fresh noise. Figure 10 shows the same three steps for any test polynomial; there the result is $v_1 = v_{\bar{\varphi}}$, the entry for slice 1 (phases close to $\frac{1}{8}$). With the gate test polynomial every v_j is μ, so any $\bar{\varphi} \in \{0, 1, 2, 3\}$ would give $+\mu$ and any $\bar{\varphi} \in \{4, \ldots, 7\}$ would give $-\mu$.

3.5. Step 4: Sample Extraction

Blind rotation gives a TRLWE ciphertext, which encrypts a whole table, but we only want its constant coefficient, and the next gate expects an LWE ciphertext. Luckily, one coefficient of a TRLWE ciphertext can be read as an LWE ciphertext without any computation. Write the accumulator as (a, b). The constant coefficient of its phase $b - a \cdot s'$ is

$$b_0 - a_0 s'_0 - \sum_{j=1}^{N-1} a_{N-j} s'_j,$$

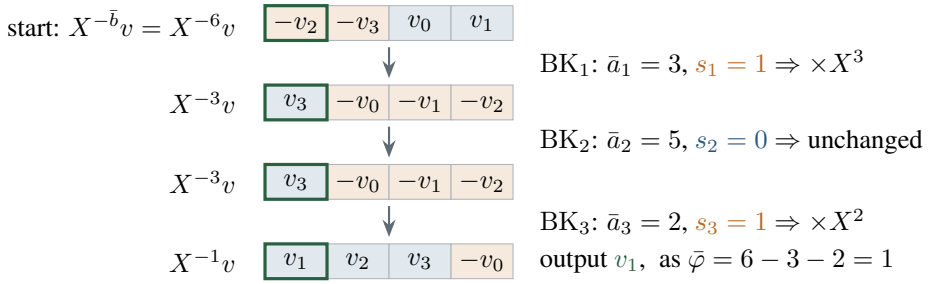


Figure 10. The blind rotation of Example 5 ($n = 3$, $N = 4$, $s = (1, 0, 1)$, $\bar{b} = 6$, $\bar{\mathbf{a}} = (3, 5, 2)$) for any test polynomial $v = v_0 + v_1X + v_2X^2 + v_3X^3$. Each row is the message of the accumulator (the coefficients of the polynomial it encrypts) after one CMux. The server does the same three CMux operations whatever s is; only the key decides which of them rotate. The final constant coefficient is $v_{\bar{\varphi}} = v_1$. With the gate test polynomial ($v_j = \mu$), the rows are the ones in the table of the example.

because, in the product $a \cdot s'$, the pairs of degrees that add up to N end up in the constant term with their sign changed ($X^N = -1$). This is the phase of the LWE ciphertext $\big((a_0, -a_{N-1}, \ldots, -a_1),\, b_0\big)$ under the key formed by the coefficients of s', seen as a vector of N bits (Figure 11). Extraction only reorders coefficients and changes some signs, so it adds no error.

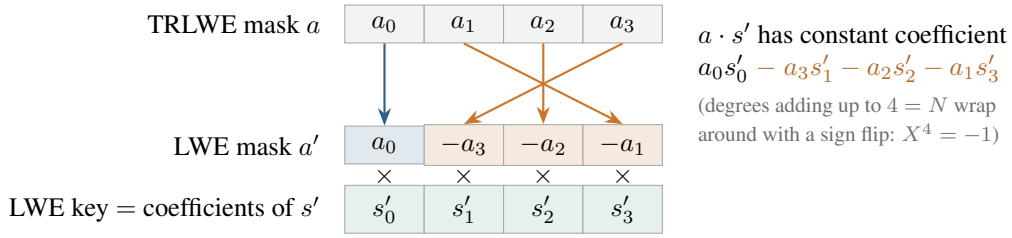


Figure 11. Sample extraction for $N = 4$. In the constant coefficient of $a \cdot s'$, each s'_j is multiplied by a_{N-j}, with a minus sign for $j \geq 1$. Reordering the mask and changing these signs gives an LWE ciphertext (a', b_0) whose phase, under the coefficients of s', is the constant coefficient of the TRLWE phase. Nothing is computed and no noise is added.

3.6. Step 5: Key Switching

Why it is needed. The extracted ciphertext has the right message but the wrong format: it is encrypted under s' and has size $N = 1024$, while the inputs of every gate are encrypted under s and have size $n = 630$. Without a conversion, the result could not be used by another gate, because the bootstrapping key encrypts the bits of s, not the bits of s'.

The idea. We want to keep the phase $b - \sum_{i=1}^{N} a_i s'_i$. If the server had LWE encryptions under s of the products $a_i s'_i$, it could subtract all of them from the trivial ciphertext $(\mathbf{0}, b)$ and get an encryption under s with the same phase. It cannot store an encryption for every possible value of a_i, but, as in the gadget decomposition, it can work digit by digit. We write each a_i with t digits in base $B_{\text{ks}} = 2^{\text{basebit}}$, $a_i \approx \sum_{j=1}^{t} a_{i,j} B_{\text{ks}}^{-j}$ with $a_{i,j} \in$

$\{0, \dots, B_{\text{ks}} - 1\}$. The *key-switching key* contains, for every coefficient i, level j and digit value v, the LWE ciphertext $\text{KSK}_{i,j,v} = \text{LWE}_s\big(v\, s'_i / B_{\text{ks}}^j\big)$. Then

$$c' = (\mathbf{0}, b) - \sum_{i=1}^{N} \sum_{j=1}^{t} \text{KSK}_{i,j,a_{i,j}}$$

has phase $b - \sum_i s'_i \sum_j a_{i,j} B_{\text{ks}}^{-j} \approx b - \sum_i a_i s'_i$ under s, plus the errors of the ciphertexts we subtracted. Since the table has an entry for every digit value, key switching only picks entries and subtracts them, with no multiplication at all.

Example 6 (One coefficient)**.** With $B_{\text{ks}} = 4$ and $t = 8$, the coefficient $a_i = 0.7$ has base-4 digits $(2, 3, 0, 3, 0, 3, 0, 3)$, because $0.7 = 0.2303\ldots$ in base 4. Its part of the key switching costs five subtractions, one for each nonzero digit (Figure 12). Doing the same for all $N = 1024$ coefficients completes the key switching.

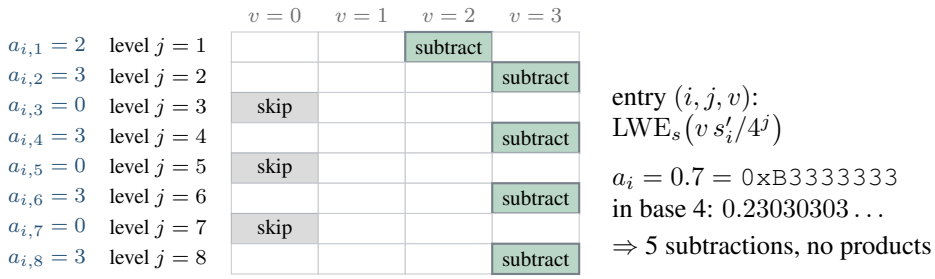


Figure 12. Key switching for one coefficient a_i of the extracted ciphertext. For each of the $t = 8$ digit positions j, the table has an LWE encryption under s of $v \cdot s'_i / 4^j$ for every possible digit v. The base-4 digits of a_i choose one entry per position; the chosen entries are subtracted from $(0, b)$, and zero digits are skipped. Doing this for all $N = 1024$ coefficients replaces the part $\sum_i a_i s'_i$ of the phase by the same value computed under the key s.

Algorithm 1 puts the five steps together.

Algorithm 1 Gate bootstrapping of an LWE ciphertext $c = (\mathbf{a}, b)$ under s

Require: $\mu \in \mathbb{T}$; $\text{BK}_i = \text{TRGSW}_{s'}(s_i)$; KSK
Ensure: LWE ciphertext under s of $+\mu$ if $\varphi_s(c) \in (0, \frac{1}{2})$, of $-\mu$ otherwise

$\bar{b} \leftarrow \lfloor 2Nb \rceil$; $\quad v \leftarrow \mu\,(1 + X + \cdots + X^{N-1})$ ▷ steps 1 and 2
$\text{ACC} \leftarrow (0,\ X^{-\bar{b}} \cdot v)$ ▷ trivial TRLWE ciphertext
for $i = 1, \dots, n$ **do** ▷ step 3: blind rotation
  $\bar{a}_i \leftarrow \lfloor 2Na_i \rceil$
  if $\bar{a}_i \neq 0$ **then**
    $\text{ACC} \leftarrow \text{ACC} + \text{BK}_i \boxdot \big((X^{\bar{a}_i} - 1) \cdot \text{ACC}\big)$ ▷ CMux, Eq. (4)
  end if
end for
$(a, b) \leftarrow \text{ACC}$; $\quad c' \leftarrow \big((a_0, -a_{N-1}, \dots, -a_1),\ b_0\big)$ ▷ step 4: key s'
return $\text{KeySwitch}(c', \text{KSK})$ ▷ step 5: key s, size n

3.7. Why the Noise Is Reset

Where does the error of the output come from? Blind rotation starts from a trivial ciphertext with *no* noise, so all the error it collects comes from the bootstrapping key, through the n external products. Key switching then adds the errors of the table entries it subtracts. The error of the input does not enter any of these operations: it only affects which slice $\bar{\varphi}$ is chosen in step 1. So, as long as the input error plus the modulus-switching error keeps the phase in the correct half of the circle, the output is a correct encryption, and its error depends only on the evaluation key. Section 5.5 shows this with a deliberately noisy input.

To put numbers on it, each of the n CMux steps adds the error of one external product (Proposition 1), and key switching adds the error of up to tN table entries plus the rounding of each a_i to t digits:

$$\vartheta_{\mathrm{BR}} \le n\big(2\ell N\beta^2\vartheta_{\mathrm{BK}} + (1+N)\varepsilon^2\big), \tag{5}$$

$$\vartheta_{\mathrm{KS}} \le tN\vartheta_{\mathrm{KSK}} + N\cdot 2^{-2(t\cdot\mathrm{basebit}+1)}. \tag{6}$$

With the parameters of Table 4, $\vartheta_{\mathrm{BR}} \le 1.4\cdot 10^{-5}$ and $\vartheta_{\mathrm{BR}} + \vartheta_{\mathrm{KS}} \le 2.2\cdot 10^{-5}$, which is a standard deviation below 0.005. These bounds have the same form as the analysis of [Chillotti et al. 2020]; Section 5.5 compares them with our measurements.

3.8. From a Sign Function to Logic Gates

With the gate test polynomial, the bootstrapping returns $+\frac{1}{8}$ (true) when the phase of its input is in the upper half of the circle, and $-\frac{1}{8}$ (false) when it is in the lower half. To compute a gate with two inputs, it is therefore enough to build, from the two inputs, a ciphertext whose phase is in the upper half exactly when the output of the gate should be true. Linear combinations are free (Section 2.3), so we only need to choose the right one.

NAND. Take $(\mathbf{0}, \frac{1}{8}) - c_x - c_y$, the trivial ciphertext of $\frac{1}{8}$ minus the two inputs. Since the inputs are $\pm\frac{1}{8}$, its phase is

$$\tfrac{1}{8} - (\pm\tfrac{1}{8}) - (\pm\tfrac{1}{8}) \in \big\{ \underbrace{\tfrac{3}{8}}_{(0,0)},\ \underbrace{\tfrac{1}{8}}_{(0,1),(1,0)},\ \underbrace{-\tfrac{1}{8}}_{(1,1)} \big\},$$

which is in the upper half for every input except $(1,1)$. This is the truth table of NAND (Figure 13a). All these points are at distance $\frac{1}{8}$ from the closest boundary (0 or $\frac{1}{2}$). This distance is the *margin*: the input errors plus the modulus-switching error must stay below it for the gate to give the right answer.

Example 7 (One NAND gate)**.** Let c_x and c_y both encrypt true, with phases $\frac{1}{8}+e_x$ and $\frac{1}{8}+e_y$. The combination $(\mathbf{0}, \frac{1}{8}) - c_x - c_y$ has phase $\frac{1}{8} - \frac{1}{8} - \frac{1}{8} - (e_x+e_y) = -\frac{1}{8} - (e_x+e_y)$, which is in the lower half as long as $|e_x+e_y|$ plus the modulus-switching error is less than $\frac{1}{8}$. So the bootstrapping returns an encryption of $-\frac{1}{8}$, that is, false, with fresh noise that no longer depends on e_x and e_y.

Other gates. The same reasoning gives AND, OR and NOR, by changing the constant and the signs (Table 1). XOR needs one more idea. The phase of $c_x + c_y$ is $-\frac{1}{4}$, 0 or $+\frac{1}{4}$, and the middle value, which corresponds to different inputs, is exactly on the boundary. Doubling fixes this: $2(c_x + c_y)$ has phase $-\frac{1}{2}$, 0 or $+\frac{1}{2}$, and since $-\frac{1}{2}$ and $+\frac{1}{2}$ are the same point of the torus, the two cases with equal inputs now fall on the same point. Adding $\frac{1}{4}$ moves them to $-\frac{1}{4}$ and the case with different inputs to $+\frac{1}{4}$ (Figure 13b). Doubling also doubles the input errors, but the margin doubles too. NOT does not need a bootstrapping at all: $-c$ swaps $\pm\frac{1}{8}$ and only changes the sign of the error.

Table 1. Linear combination computed before the bootstrapping of each gate.

Gate	Combination	Margin	Gate	Combination	Margin
NAND	$(\mathbf{0}, +\frac{1}{8}) - c_x - c_y$	$\frac{1}{8}$	AND	$(\mathbf{0}, -\frac{1}{8}) + c_x + c_y$	$\frac{1}{8}$
OR	$(\mathbf{0}, +\frac{1}{8}) + c_x + c_y$	$\frac{1}{8}$	NOR	$(\mathbf{0}, -\frac{1}{8}) - c_x - c_y$	$\frac{1}{8}$
XOR	$(\mathbf{0}, +\frac{1}{4}) + 2(c_x + c_y)$	$\frac{1}{4}$	XNOR	$(\mathbf{0}, -\frac{1}{4}) - 2(c_x + c_y)$	$\frac{1}{4}$

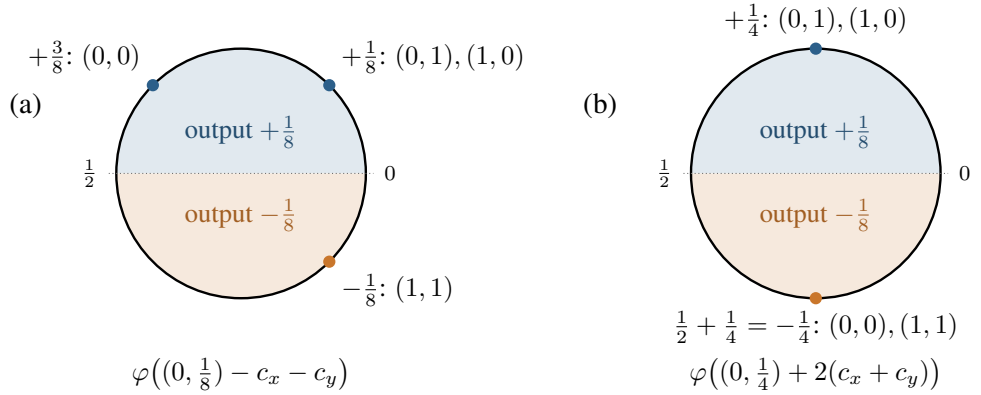


Figure 13. Phases at the input of the bootstrapping, without noise. (a) NAND: every point is at distance $\frac{1}{8}$ from the closest boundary (0 or $\frac{1}{2}$); this distance is the noise margin. (b) XOR: with c_x+c_y, different inputs would fall on the boundary. After doubling, the two cases with equal inputs fall on the same point $\pm\frac{1}{2}$, and adding $\frac{1}{4}$ separates the two groups with a margin of $\frac{1}{4}$.

MUX. The multiplexer $\mathrm{MUX}(s, x, y)$, "x if s is true, otherwise y", can be written as $(s \wedge x) \vee (\neg s \wedge y)$, which would need three bootstrappings. But the two ANDs are never true at the same time, so their OR can be replaced by a simple sum. We compute both ANDs with bootstrappings *without* key switching (so their outputs are under s'), add the two results, which gives $-\frac{1}{4}$ or 0, add $\frac{1}{8}$ to get $-\frac{1}{8}$ or $+\frac{1}{8}$, and do a single key switching at the end (Table 2). A MUX therefore costs two blind rotations and one key switching.

3.9. An Encrypted Adder

Every gate gives back a ciphertext in the same format as its inputs and with fresh noise, so gates can be connected into circuits of any size and depth. As a complete example, we built a circuit that adds two encrypted integers.

Table 2. The MUX gate on all inputs. u_1 and u_2 are the results of the two bootstrappings without key switching. At most one of them is $+\frac{1}{8}$, so $u_1 + u_2 + \frac{1}{8}$ is always $\pm\frac{1}{8}$.

s	x	y	$u_1 = \mathrm{BR}(s \wedge x)$	$u_2 = \mathrm{BR}(\neg s \wedge y)$	$u_1 + u_2 + \frac{1}{8}$	output
0	0	0	$-\frac{1}{8}$	$-\frac{1}{8}$	$-\frac{1}{8}$	$0 = y$
0	0	1	$-\frac{1}{8}$	$+\frac{1}{8}$	$+\frac{1}{8}$	$1 = y$
0	1	0	$-\frac{1}{8}$	$-\frac{1}{8}$	$-\frac{1}{8}$	$0 = y$
0	1	1	$-\frac{1}{8}$	$+\frac{1}{8}$	$+\frac{1}{8}$	$1 = y$
1	0	0	$-\frac{1}{8}$	$-\frac{1}{8}$	$-\frac{1}{8}$	$0 = x$
1	0	1	$-\frac{1}{8}$	$-\frac{1}{8}$	$-\frac{1}{8}$	$0 = x$
1	1	0	$+\frac{1}{8}$	$-\frac{1}{8}$	$+\frac{1}{8}$	$1 = x$
1	1	1	$+\frac{1}{8}$	$-\frac{1}{8}$	$+\frac{1}{8}$	$1 = x$

Encrypting an integer. TFHE encrypts bits, so an n-bit integer is encrypted bit by bit, as n separate LWE ciphertexts, from the least significant bit to the most significant one. For example, $x = 11 = 1011_2$ is stored as the four ciphertexts $(\mathrm{Enc}(1), \mathrm{Enc}(1), \mathrm{Enc}(0), \mathrm{Enc}(1))$.

Adding by hand in binary. The circuit follows the method we learn in school: add the two numbers column by column, from right to left, and carry a 1 to the next column when the column sum is 2 or more. For $11 + 6$:

carries	1	1	1	0	
$x = 11$		1	0	1	1
$y = 6$		0	1	1	0
$x + y = 17$	1	0	0	0	1

In each column i there are three input bits, x_i, y_i and the incoming carry c_i, and two output bits, the sum bit sum_i and the outgoing carry c_{i+1}. A circuit that computes one column is called a *full adder*, and Table 3 gives its truth table.

Table 3. Truth table of a full adder, with the formulas used by our circuit ($p = x \oplus y$).

x_i	y_i	c_i	sum_i	c_{i+1}	which case
0	0	0	0	0	$x_i = y_i$: carry is $x_i = 0$
0	0	1	1	0	$x_i = y_i$: carry is $x_i = 0$
0	1	0	1	0	$x_i \neq y_i$: carry is $c_i = 0$
0	1	1	0	1	$x_i \neq y_i$: carry is $c_i = 1$
1	0	0	1	0	$x_i \neq y_i$: carry is $c_i = 0$
1	0	1	0	1	$x_i \neq y_i$: carry is $c_i = 1$
1	1	0	0	1	$x_i = y_i$: carry is $x_i = 1$
1	1	1	1	1	$x_i = y_i$: carry is $x_i = 1$

From the truth table to gates. The sum bit is 1 when an odd number of the three inputs are 1, so $sum_i = x_i \oplus y_i \oplus c_i$. For the carry, the last column of Table 3 shows a simple rule. When $x_i = y_i$, the outgoing carry is equal to x_i, whatever the incoming carry is: two 1s always produce a carry and two 0s never do. When $x_i \neq y_i$, the column sum is $1 + c_i$, so the outgoing carry is just the incoming one. This is a choice controlled by $p_i = x_i \oplus y_i$, so

$$sum_i = p_i \oplus c_i, \qquad c_{i+1} = \mathrm{MUX}(p_i;\, c_i,\, x_i).$$

The value p_i is used twice, so one column needs only two XOR gates and one MUX (Figure 14), that is, four blind rotations. Adding two n-bit numbers takes n columns and $4n$ blind rotations, and the result has $n+1$ bits, the last one being the final carry. The first carry, $c_0 = 0$, is a trivial ciphertext of false, which the server can create without any key.

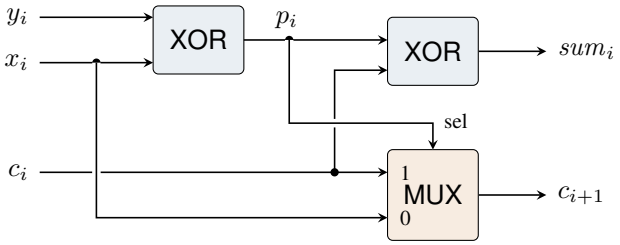


Figure 14. One column of the adder (a full adder): $sum_i = p_i \oplus c_i$ and $c_{i+1} = \mathrm{MUX}(p_i;\, c_i,\, x_i)$, with $p_i = x_i \oplus y_i$. When $x_i = y_i$ the outgoing carry is x_i, whatever c_i is; otherwise the incoming carry c_i goes through. Each column uses two XOR gates and one MUX, that is, four blind rotations.

Example 8 ($11 + 6$ on encrypted bits)**.** With $x = 11$, $y = 6$ and $c_0 = 0$, the circuit goes through the four columns as follows (bits from the least significant one):

column i	x_i	y_i	c_i	$p_i = x_i \oplus y_i$	$sum_i = p_i \oplus c_i$	$c_{i+1} = \mathrm{MUX}(p_i; c_i, x_i)$
0	1	0	0	1	1	$c_0 = 0$
1	1	1	0	0	0	$x_1 = 1$
2	0	1	1	1	0	$c_2 = 1$
3	1	0	1	1	0	$c_3 = 1$

The output bits are $(sum_0, \ldots, sum_3, c_4) = (1, 0, 0, 0, 1)$, i.e., $10001_2 = 17$, the same result as by hand. On encrypted data, every entry of this table is a ciphertext, and the server never sees any of these bits. Because of this, the server cannot stop early or skip a column when there is no carry: it always evaluates the same 12 gates, whatever the inputs are.

4. Implementation

4.1. Organization

The library is made of eleven C++ header files with `inline` functions, so each test or program is a single source file and nothing has to be linked. The headers form a chain in which each layer includes the previous ones (Figure 15). Each header has its own unit test, which checks it against a simple, independent reference before the next layer is built on top of it (Table 5). The library has 526 lines of code, not counting comments and blank lines. We use the parameters of the 128-bit gate-bootstrapping set of the original library (Table 4), so that the comparison in Section 5 is fair.

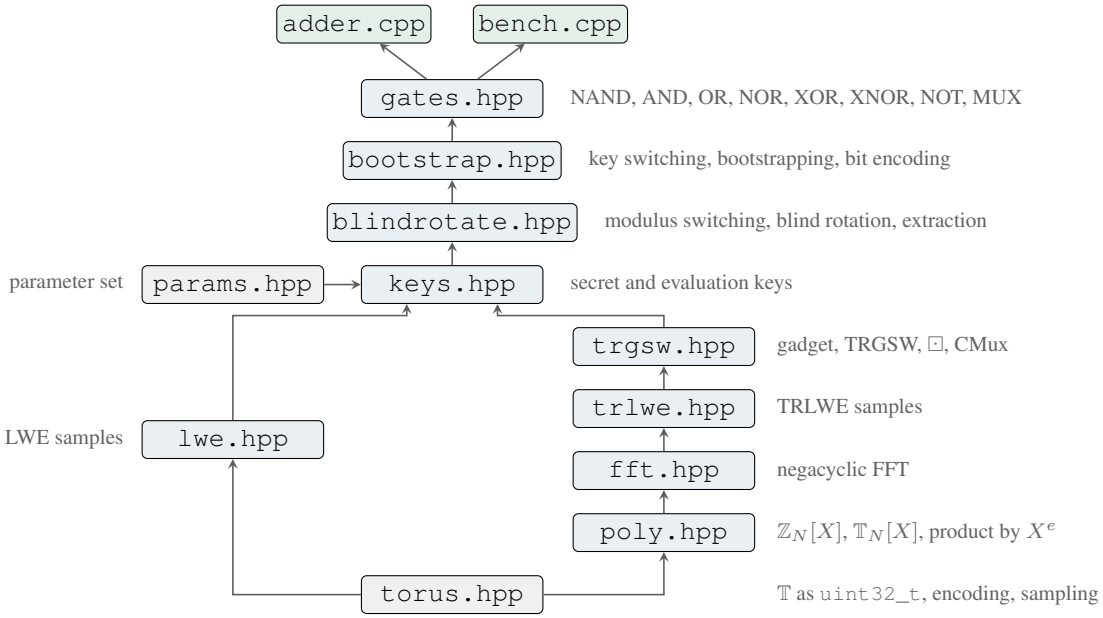


Figure 15. Structure of the library. Each box is a header file in `src/` with its own unit test in `tests/`; an arrow goes from a header to the headers that include it. The programs in `apps/` only use `gates.hpp`.

Table 4. Parameters (the same as the 128-bit set of the original library).

Symbol	Value	Role
n	630	size of the LWE key; number of external products per bootstrapping
N	1024	degree of the polynomials; the phase is rounded to $2N$ positions
ℓ, B_g	3, 2^7	number of digits and base of the gadget decomposition
t, B_{ks}	8, 2^2	number of digits and base of the key switching
σ_{LWE}	2^{-15}	noise of LWE ciphertexts and of KSK
σ_{BK}	2^{-25}	noise of the TRGSW ciphertexts of BK

Table 5. Unit tests. `make test` runs them in order and stops at the first failed check.

Test	What it checks
`test01_torus`	conversions, wrap-around, decoding in $\mathbb{Z}_8$, Gaussian noise
`test02_lwe`	decryption, additions, noise of a sum growing by $\sqrt{2}$
`test03_poly`	$X^N = -1$, $X^{2N} = 1$, rotation against the direct product
`test04_fft`	FFT and back; products identical to the direct ones
`test05_trlwe`	TRLWE decryption, addition, rotation by X^{100}
`test06_trgsw`	digit range and error, $\text{TRGSW}(m) \boxdot d$, CMux
`test07_keys`	key sizes; each BK_i selects correctly; KSK contents
`test08_blindrotate`	rotation without noise, sign function, four-entry table
`test09_bootstrap`	key switching, bootstrappings in a row, noise reduction
`test10_gates`	truth tables, 50 NAND gates in a row, majority circuit

4.2. Torus Arithmetic

A computer cannot store real numbers exactly, so implementations use a discretized torus: the 32-bit integer t stands for the real number $t/2^{32}$. With this choice the reduction modulo 1 costs nothing, because unsigned 32-bit arithmetic already wraps around modulo 2^{32}.

Example 9 (Torus arithmetic in 32 bits)**.** $0.75 + 0.5 = 1.25 \equiv 0.25$ becomes $\texttt{0xC0000000} + \texttt{0x80000000} = \texttt{0x1 4000 0000}$, and keeping the 32 low bits gives $\texttt{0x40000000} = 2^{30}$, which is 0.25. In the same way, $3 \times 0.375 \equiv 0.125$ becomes $3 \times \texttt{0x60000000} = \texttt{0x1 2000 0000} \mapsto \texttt{0x20000000}$.

A torus element is therefore a `uint32_t`, and three rules of C++ do all the work. First, unsigned arithmetic is defined modulo 2^{32}, which is exactly what the torus needs. Signed overflow, on the other hand, is undefined behavior, and an optimizing compiler may break code that relies on it. Second, in an expression like `k * t` with a signed `k`, the integer is converted to unsigned before the product, which still gives the right result modulo 2^{32} when k is negative. Third, `static_cast<int32_t>(t)` reads the same bits as a signed integer, that is, as the representative of t in $[-\frac{1}{2}, \frac{1}{2})$. We use this to measure noise (a small negative error reads as -0.001 and not as 0.999) and to decrypt a bit (by the sign of its phase). The bit $\mu = \frac{1}{8}$ of the gates is the 32-bit value `0x20000000`, so the gate test polynomial is built with the single line `TorusPoly testv(N, MU)`.

4.3. Fast Polynomial Products: the Negacyclic FFT

Why we need it. Every external product multiplies $2\ell = 6$ digit polynomials by the two polynomials of a TRGSW row, so it computes 12 products in $\mathbb{Z}_N[X] \times \mathbb{T}_N[X]$. A gate does about $n = 630$ external products, which makes about 7500 polynomial products per gate. If we compute each product term by term, it costs $N^2 \approx 10^6$ multiplications, and a single gate would take several seconds. Instead, we use the well-known idea of multiplying polynomials through their values, adapted to polynomials modulo $X^N + 1$. We explain it in four steps.

Step 1: multiply values instead of coefficients. A polynomial of degree less than N is completely determined by its values at N different points, and the value of a product is the product of the values, $(a \cdot b)(x) = a(x)\, b(x)$. So we can evaluate both factors at enough points, multiply the values one by one, and then recover the coefficients of the product from its values (interpolation).

Example 10 (Multiplying through values)**.** Take $a(x) = 1 + 2x$ and $b(x) = 3 + x$. Their product has degree 2, so three points are enough; we use 0, 1 and -1. The values are $a = (1, 3, -1)$ and $b = (3, 4, 2)$, and multiplying them one by one gives $(3, 12, -2)$. The only polynomial of degree 2 with these values is $3 + 7x + 2x^2$, which is indeed $(1 + 2x)(3 + x)$.

Multiplying the values costs only N operations. Evaluating and interpolating in a direct way would still cost about N^2 operations, but with the fast Fourier transform (FFT) [Cooley and Tukey 1965] both cost about $N \log N$, if the points are well chosen.

Step 2: choose points where the reduction is automatic. We do not want the full product $a \cdot b$, which has degree up to $2N - 2$, but its remainder modulo $X^N + 1$. If we

evaluate at a root ω of $X^N + 1$, the difference disappears: since $\omega^N = -1$, the product and its remainder have the same value at ω, because they differ by a multiple of $X^N + 1$. The N roots of $X^N + 1$ are the points $e^{i\pi(2k+1)/N}$ of the unit circle, the odd multiples of π/N (Figure 16). Moreover, a polynomial with real coefficients takes conjugate values at conjugate points, so we only need one root of each conjugate pair. That leaves $N/2$ complex values, and we use the roots $\omega_k = e^{i\pi(4k+1)/N}$ for $k = 0, \ldots, N/2 - 1$.

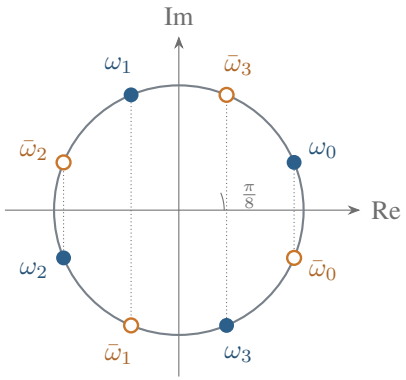


Figure 16. The roots of $X^8 + 1$, at the odd multiples of $\pi/8$ on the unit circle. The negacyclic FFT evaluates a polynomial at the four points $\omega_k = e^{i\pi(4k+1)/N}$ (filled). The other four roots (hollow, joined by dotted lines) are their complex conjugates, where a polynomial with real coefficients takes conjugate values, so they do not need to be computed.

Example 11 (A negacyclic product through values, $N = 4$)**.** Let $a = 1 + X$ and $b = X^3$. Computed directly, $a \cdot b = X^3 + X^4 = -1 + X^3$, so the result is $r = [-1, 0, 0, 1]$. The two points are $\omega_0 = e^{i\pi/4}$ and $\omega_1 = e^{i5\pi/4}$:

	$\omega_0 = e^{i\pi/4}$	$\omega_1 = e^{i5\pi/4}$
$a(\omega) = 1 + \omega$	$1.707 + 0.707i$	$0.293 - 0.707i$
$b(\omega) = \omega^3$	$-0.707 + 0.707i$	$0.707 - 0.707i$
$a(\omega)\, b(\omega)$	$-1.707 + 0.707i$	$-0.293 - 0.707i$
$r(\omega) = -1 + \omega^3$	$-1.707 + 0.707i$	$-0.293 - 0.707i$

The products of the values are already the values of the reduced product r, because $\omega^4 = -1$ at both points.

Step 3: turn it into a standard FFT. Evaluating at the points ω_k is not yet a standard discrete Fourier transform (DFT), whose points are the roots of unity $e^{2\pi ik/M}$. Two tricks close the gap. First, since $\omega_k^{N/2} = e^{i\pi(4k+1)/2} = i$, the terms of degree j and $j + N/2$ can be grouped: $p_j\omega_k^j + p_{j+N/2}\,\omega_k^{j+N/2} = (p_j + i\,p_{j+N/2})\,\omega_k^j$. This *folds* the N real coefficients into $N/2$ complex numbers. Second, we write $\omega_k^j = e^{i\pi j/N} \cdot e^{2\pi ijk/(N/2)}$. The first factor depends only on j, and we multiply it into the folded vector (the *twist*). The second factor is a standard root of unity. Together,

$$p(\omega_k) = \sum_{j<N/2} \underbrace{\left(p_j + i\,p_{j+N/2}\right) e^{i\pi j/N}}_{z_j}\; e^{2\pi ijk/(N/2)},$$

which is the standard DFT of size $N/2$ of the folded and twisted vector z (Figure 17). This is a special case of the weighted transforms of [Crandall and Fagin 1994]. To go back from values to coefficients, we do the same steps in reverse order: inverse DFT, multiply by $e^{-i\pi j/N}$, and unfold the real and imaginary parts.

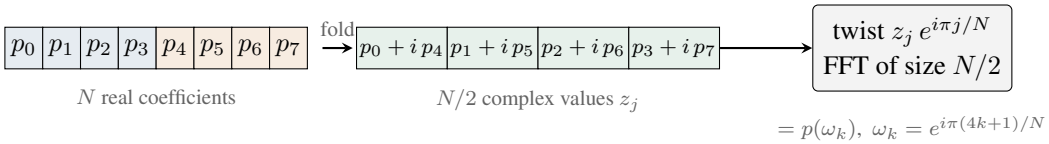


Figure 17. Forward negacyclic transform, drawn for $N = 8$. The two halves of the coefficient vector are folded into $N/2$ complex numbers, which are multiplied by the twist factors $e^{i\pi j/N}$. A standard FFT of size $N/2$ then gives the values of p at one root of $X^N + 1$ from each conjugate pair. The inverse transform does the same steps in reverse order.

Example 12 (The complete computation for $N = 4$)**.** We redo the previous example with the transform. To transform a polynomial $p = [p_0, p_1, p_2, p_3]$ we fold it into $z = (p_0 + ip_2,\ p_1 + ip_3)$, multiply the second entry by the twist $e^{i\pi/4} = 0.707 + 0.707i$, and apply a DFT of size 2, which simply returns the sum and the difference of the two entries:

	folded z	twisted	DFT: $(z_0 + z_1,\ z_0 - z_1)$
$a = [1, 1, 0, 0]$	$(1,\ 1)$	$(1,\ 0.707 + 0.707i)$	$(1.707 + 0.707i,\ 0.293 - 0.707i)$
$b = [0, 0, 0, 1]$	$(0,\ i)$	$(0,\ -0.707 + 0.707i)$	$(-0.707 + 0.707i,\ 0.707 - 0.707i)$

These are the values of a and b at ω_0 and ω_1 from the previous table. Multiplying them one by one gives $P = (-1.707 + 0.707i,\ -0.293 - 0.707i)$. To go back, the inverse DFT of size 2 returns half the sum and half the difference, $(-1,\ -0.707 + 0.707i)$; undoing the twist (multiplying the second entry by $e^{-i\pi/4}$) gives $(-1,\ i)$; and unfolding, with real parts as r_0, r_1 and imaginary parts as r_2, r_3, gives $r = [-1, 0, 0, 1]$, the same result as the direct computation.

Step 4: the FFT itself. A DFT of size M computed directly needs M^2 multiplications. The FFT of [Cooley and Tukey 1965] is faster because it splits the problem: a DFT of size M can be computed from two DFTs of size $M/2$, one on the entries with even index and one on the entries with odd index, and those can be split again, until size 1. Figure 18 shows the case $M = 4$. After putting the inputs in a special order (bit-reversed), each round combines pairs of values with a *butterfly*, $(u, v) \mapsto (u + w\,v,\ u - w\,v)$, where w is a root of unity. For our size, $M = N/2 = 512$, this means 9 rounds of 256 butterflies, that is, 2304 butterflies instead of about 260,000 multiplications. We compute all the twist factors, roots of unity and the bit-reversed order once, when the `NegFFT` object is created.

Is double precision enough? The FFT computes with `double` numbers, which have 53 bits of precision, but the result must be exact as a 32-bit integer. In every product of the scheme, one factor has small coefficients (digits with $|d| \leq 64$, or key bits) and the other has torus coefficients, read as signed integers with $|t| \leq 2^{31}$. Each coefficient of the product is a sum of $N = 2^{10}$ such terms, so its absolute value is at most $2^6 \cdot 2^{31} \cdot 2^{10} = 2^{47}$, which is less than 2^{53}. Rounding the result to the nearest integer therefore gives the exact

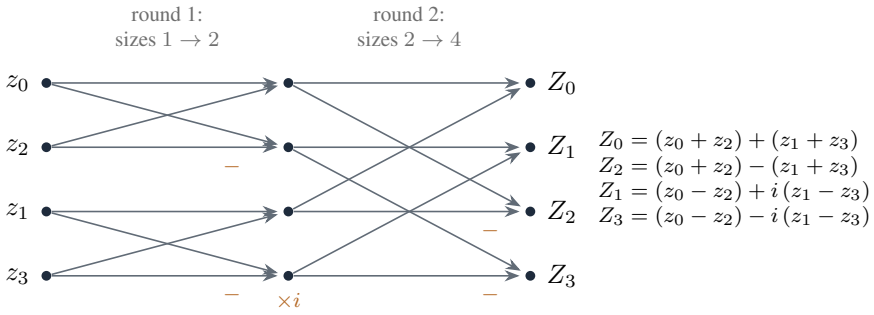


Figure 18. A DFT of size 4 computed by the FFT. The inputs are first put in bit-reversed order (z_0, z_2, z_1, z_3). Each round combines pairs of values with a *butterfly*, $(u, v) \mapsto (u + w\,v,\ u - w\,v)$, where w is a root of unity (1, or i for the last pair). A transform of size M needs $\log_2 M$ rounds of $M/2$ butterflies.

value. Our unit test confirms this: the FFT products were identical, bit by bit, to the direct ones, and $60\times$ faster ($325\,\mu$s against $5.4\,\mu$s for $N = 1024$).

A detail about complex numbers. The product of two `std::complex<double>` follows the C99 rules for infinities and NaN values, which makes the compiler unable to vectorize it and slows down the FFT loop. We use a hand-written function `cmul` with the usual formula $(x + iy)(u + iv) = (xu - yv) + i(xv + yu)$.

4.4. Signed Digits with One Addition: the Offset Trick

The problem. Every external product decomposes $2N = 2048$ coefficients into $2N\ell = 6144$ digits, so the gadget decomposition must be fast. Ordinary digits in $[0, B_g)$ would be easy: they are just groups of bits of t. But we want signed digits in $[-B_g/2, B_g/2)$, because they are half as large ($\beta = B_g/2$ instead of B_g), and the main noise term of Proposition 1 grows with β^2, so signed digits give four times less noise.

The slow way. To get signed digits from ordinary ones, we can work from the last digit to the first, like in a manual subtraction: every digit that is $B_g/2$ or larger becomes negative by subtracting B_g, and we add 1 to the digit on its left. Before that, we also have to round, using the part of t that is thrown away. Take base 10 with two digits (digits in $[-5, 5)$) and $t = 0.9551$. Rounding to two digits gives 0.96, because the discarded part, 51, is at least half. The digits are 9 and 6. The last digit, 6, is too large, so it becomes $6 - 10 = -4$ and the 9 becomes 10. Now 10 is too large, so it becomes 0 and gives a 1 to the left, which is a full turn of the circle and can be ignored. The result is $(0, -4)$: indeed $0 - 0.04 = -0.04 \equiv 0.96$. This works, but it needs a loop with tests and carries, for each of the 6144 digits.

The trick. The same result can be obtained with one addition. Add to t a constant, the *offset*, that has $B_g/2$ in every digit position plus half of the position after the last digit; read the ordinary digits of the sum; and subtract $B_g/2$ from each of them. In base 10 with two digits, the offset is 0.555. Table 6 applies the trick to three values.

Table 6. The offset trick in base 10 with two digits: add 0.555, keep the part after the decimal point, take the two digits and subtract 5 from each.

t	$t + 0.555$	digits	minus 5	value	error
0.7316	$1.2866 \to 0.2866$	$2,\ 8$	$-3,\ 3$	$-0.27 \equiv 0.73$	0.0016
0.7366	$1.2916 \to 0.2916$	$2,\ 9$	$-3,\ 4$	$-0.26 \equiv 0.74$	0.0034
0.9551	$1.5101 \to 0.5101$	$5,\ 1$	$0,\ -4$	$-0.04 \equiv 0.96$	0.0049

The first row is Example 3. The second row shows the rounding: the discarded part of 0.7366 is 66, so the value is rounded up to 0.74. The third row is the value we decomposed the slow way, and it gives the same digits $(0, -4)$, including the chain of two carries. In every row the error is at most $\varepsilon = 0.005$.

Why it works. Adding 5 to every digit and later subtracting 5 from every digit does not change the value. What changes is that the addition causes a carry exactly when a digit was 5 or more, and that is exactly when the slow way had to make the digit negative and add 1 to its left neighbor. The extra half digit at the end does the rounding in the same way: it causes a carry into the last digit exactly when the discarded part is at least half. The following lemma states this for our binary case, where a digit is a group of β_{bit} bits.

Lemma 1 (Offset trick)**.** *Let $q = 2^{32}$, $B_g = 2^{\beta_{\text{bit}}}$ with $\ell\beta_{\text{bit}} < 32$, let $h_i = 2^{32-i\beta_{\text{bit}}}$ (the integer that stands for B_g^{-i}), and*

$$\mathit{offset} = \sum_{i=1}^{\ell} \frac{B_g}{2}\, h_i + \frac{h_\ell}{2}.$$

For $t \in \mathbb{Z}_q$, let $t' = t + \mathit{offset} \bmod q$, let $u_i \in [0, B_g)$ be the i-th group of β_{bit} bits of t' from the top, i.e., $u_i = \lfloor t'/h_i \rfloor \bmod B_g$, and let $d_i = u_i - B_g/2$. Then $d_i \in [-B_g/2, B_g/2)$ and $\left| t - \sum_i d_i h_i \bmod q \right| \le h_\ell/2$, so the digits (d_i) satisfy Eq. (1).

Proof. We can write $t' = \sum_{i=1}^{\ell} u_i h_i + r$ with $0 \le r < h_\ell$. Then, modulo q, $t = t' - \mathit{offset} = \sum_i (u_i - B_g/2)\, h_i + (r - h_\ell/2) = \sum_i d_i h_i + (r - h_\ell/2)$, and $|r - h_\ell/2| \le h_\ell/2$, which is $\varepsilon = 1/(2B_g^{\ell})$ in torus units. The range of d_i follows from $u_i \in [0, B_g)$. □

In binary. With $B_g = 2^7$ and $\ell = 3$, a digit is a group of 7 bits, and the offset has 64 in each of the three groups and 2^{10} in the discarded part. Figure 19 shows the computation for $t = 0.7$: one 32-bit addition, then each digit is obtained with a shift, a mask and a subtraction. The code below is the complete decomposition of one coefficient; the offset is computed only once.

```
uint32_t t = p[j] + g.offset;              // one addition: all carries
for (int i = 0; i < g.ell; ++i) {
    uint32_t digit = (t >> (32 - (i + 1) * g.Bgbit)) & g.mask; // [0,Bg)
    out[i][j] = int32_t(digit) - int32_t(g.halfBg);     // [-Bg/2,Bg/2)
}
```

The same idea of "adding half before cutting" is used in three other places: to decode messages, in modulus switching (where 2^{31} is added to $2N \cdot t$, computed in 64

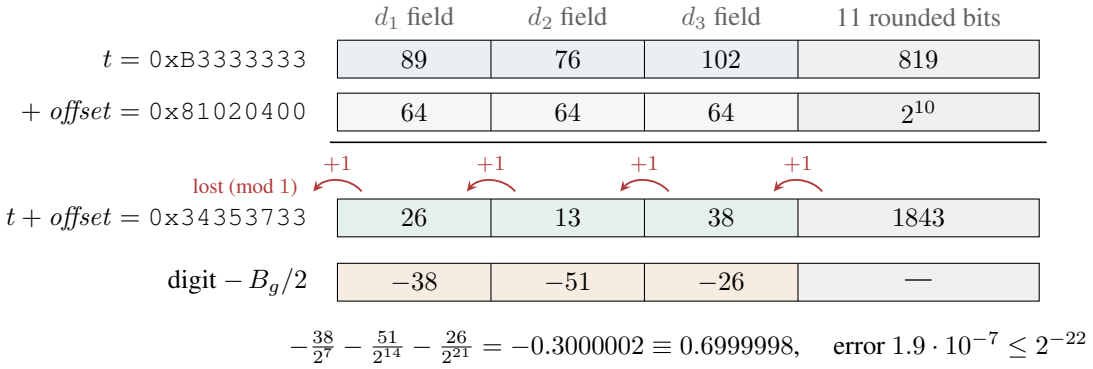


Figure 19. The offset trick for $t = 0.7$, with $B_g = 2^7$ and $\ell = 3$. The offset has 64 in each 7-bit digit group and half of the last digit (2^{10}) in the part that is thrown away. After one 32-bit addition, each digit is obtained with a shift and a mask, and subtracting 64 gives signed digits in $[-64, 64)$. The ordinary digits of t are $(89, 76, 102)$; the carries of the addition (red) make each digit that is 64 or more negative and add 1 to the digit on its left.

bits, before dropping the 32 low bits), and in key switching, where a half digit is added to each a_i so that its t digits are rounded instead of truncated.

4.5. External Product in the Frequency Domain

The same rows of the bootstrapping key are used in every gate, so we store them already transformed by the FFT: each TRGSW is kept as 2ℓ rows of two polynomials, each one as $N/2$ complex values. We also add up the products of Eq. (3) while still in the frequency domain, where addition and multiplication are done value by value (Figure 20). So an external product costs $2\ell = 6$ forward transforms (one per digit polynomial), $4\ell = 12$ value-by-value products, and only 2 inverse transforms, about $8 \times 630 \approx 5000$ FFTs of size 512 per gate. The price is memory: a complex `double` takes 16 bytes for every two coefficients, twice as much as the coefficients themselves, so the bootstrapping key takes 59.1 MiB. Blind rotation also skips the steps where $\bar{a}_i = 0$, since $X^0 - 1 = 0$.

4.6. Key-Switching Table

The key-switching key stores one ciphertext for every possible digit value, so that key switching needs no multiplications: at most $Nt = 8192$ subtractions of vectors of $n + 1$ torus elements, which take about 0.8 ms. The $N \cdot t \cdot B_{\text{ks}} = 32768$ ciphertexts, 78.9 MiB in total, are stored in a single array, with entry (i, j, v) at position $(i\,t + j)\,B_{\text{ks}} + v$, so that the ciphertexts used for one coefficient are next to each other in memory. Entries with $v = 0$ are never used; they hold a trivial encryption of zero so that the indexing stays simple.

5. Experimental Evaluation

The project proposal set four criteria for the comparison with the original library: correctness (does every gate decrypt to the right bit?), performance (how long does a gate take, and where is the time spent?), memory (how large are the keys?) and noise (does the measured error match the theory?). This section answers them in this order, after describing how we ran the experiments.

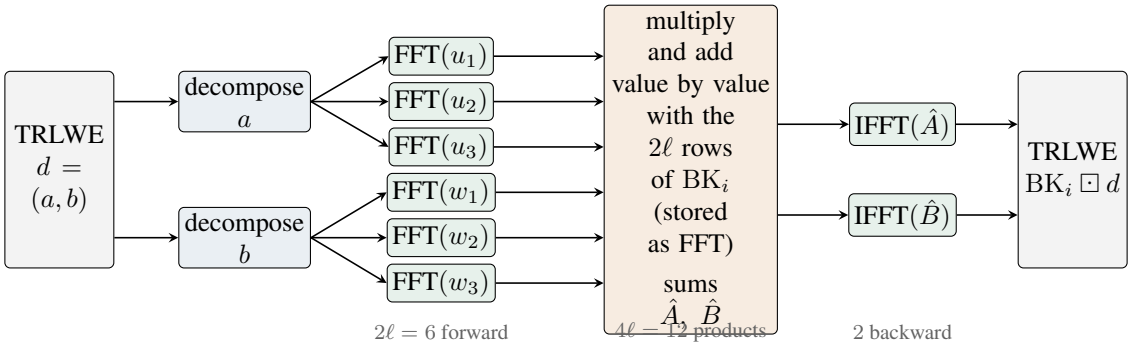


Figure 20. One external product in our implementation. The digit polynomials of both components are transformed once and multiplied by the rows of the TRGSW ciphertext, which were transformed when the key was created. The products are added up in the frequency domain, so each external product needs only two inverse transforms. Blind rotation does one external product for each nonzero $\bar{a}_i$, about $n = 630$ per gate.

5.1. Setup

We ran all measurements on a laptop with an Intel Core Ultra 7 256V processor, running Linux, on a single thread fixed to one performance core, with the power profile set to performance. Our code was compiled with GCC 15.3.1 and the options `-O3 -march=native`. The original library [Chillotti et al. 2016b] was compiled with its own optimized configuration in two versions that differ only in the FFT: *spqlios-fma*, written in assembly with AVX2 and FMA instructions, and *nayuki-portable*, written in plain C. Both use the same 128-bit parameters as ours (Table 4). The programs `apps/bench.cpp` and `compare/bench_original.cpp` do the same measurements with the two libraries: the time to generate the keys, the average time of 500 NAND gates, the error of 500 gates with random inputs (phase minus the expected $\pm\frac{1}{8}$), and the number of decryption errors. Each run creates new keys. We did 5 runs of our implementation and 3 of each version of the original library, and we report the range of the results.

5.2. Correctness

All ten unit tests pass. The adder of Section 3.9 gave the right sum in every run: $173{+}94 = 267$ (8 bits) in 0.72 s, $40000{+}30000 = 70000$ (16 bits) in 1.43 s, and $9999999{+}7777777 = 17777776$ (24 bits) in 2.16 s. The time grows linearly with the number of bits, as expected from four blind rotations per bit. Across the benchmark and noise experiments, 12,500 gates with random inputs gave no decryption error, and neither did the original library in its 3,000 gates. This agrees with the noise margin: the input of a NAND is the sum of two bootstrapping outputs plus the modulus-switching error, with a standard deviation of about 0.005, while the margin is $\frac{1}{8} = 0.125$, more than 20 standard deviations away.

5.3. Performance

Table 7 and Figure 21 summarize the comparison. Our implementation computes a NAND gate in 22.9 ms, which is $2.3\times$ slower than the original library with the assembly FFT and $4.9\times$ faster than the same library with the portable FFT. Since the rest of

the algorithm is the same in all three cases, this shows that the speed of TFHE depends mostly on the quality of the FFT. Key generation is also faster in our implementation. A likely reason is that the original library creates and keeps the bootstrapping key both as coefficients and in the frequency domain, while ours keeps only the second form.

Table 7. Comparison with the original library (range over the runs, 500 gates per run, one thread). Memory is the peak memory used by the benchmark program.

	This work	Original library	
		spqlios-fma	nayuki-portable
Time per NAND gate (ms)	22.1–23.8	9.7–9.8	107.8–122.1
Key generation (s)	0.21–0.22	0.53	0.88–0.94
Output error variance (10^{-5})	0.78–0.92	1.00–1.12	1.00–1.16
Largest |mean| of the output error	$1.2 \cdot 10^{-3}$	$2.0 \cdot 10^{-4}$	$1.9 \cdot 10^{-4}$
Decryption errors	0 in 2,500	0 in 1,500	0 in 1,500
Peak memory (MiB)	143	253	253

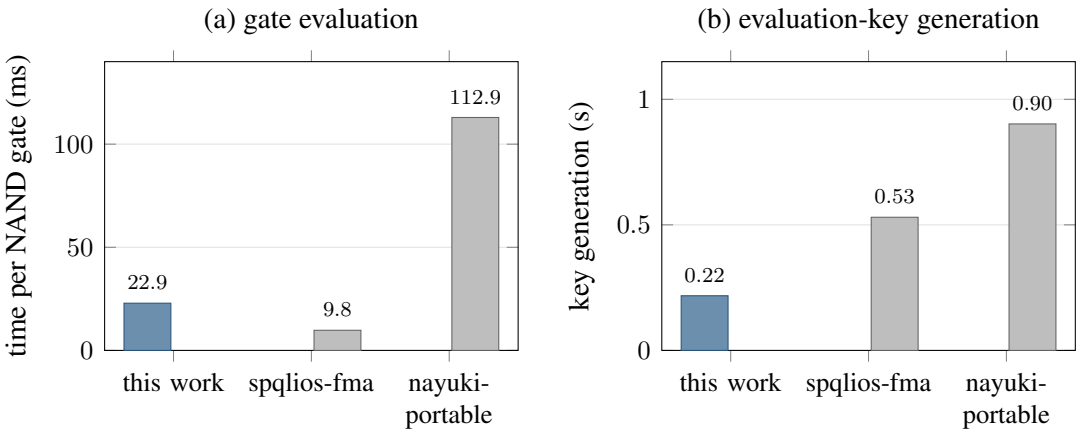


Figure 21. Time per gate and key generation time, on one thread, with the same parameters and on the same machine (average over the runs; the ranges are in Table 7). The original library is shown with its two FFT versions; the rest of its code is the same in both.

Where the time goes. To find the bottlenecks, we measured the time of each operation used by the blind rotation on its own, and multiplied it by the number of times it runs in one gate (Figure 22). Blind rotation takes 96.5% of the gate time and key switching the rest. The FFTs alone, forward and backward, take 42% of the gate, and the value-by-value products another 22%, so the work in the frequency domain is 63% of the time. The gadget decomposition, which with the offset trick is only a few integer operations per coefficient, takes only 7%.

The measurements also showed a cost that does not appear in the description of the algorithm: *memory traffic*. Blind rotation reads a different TRGSW ciphertext at each step, so every gate reads the whole 59.1 MiB bootstrapping key, which is much larger than the processor caches. One external product takes 21.8 µs when it reuses the same key

ciphertext (which then stays in the cache), but 33.6 µs when consecutive calls go through the key as in the blind rotation. A single value-by-value product is $5.6\times$ slower when its TRGSW row comes from main memory (0.64 µs instead of 0.11 µs), so Figure 22 counts the products with the slower time. Over the n steps, this difference is about 34% of the gate time. The remaining 25% ("other") is the part that no single operation explains: the temporary vectors created in every external product (about 3 µs per call, measured as the difference between a full external product that stays in the cache and the sum of its parts) and other waits for memory. These results agree with the comparison above: the $2.3\times$ difference to the original library comes from the work in the frequency domain and from how it accesses memory, which the original library implements in optimized assembly.

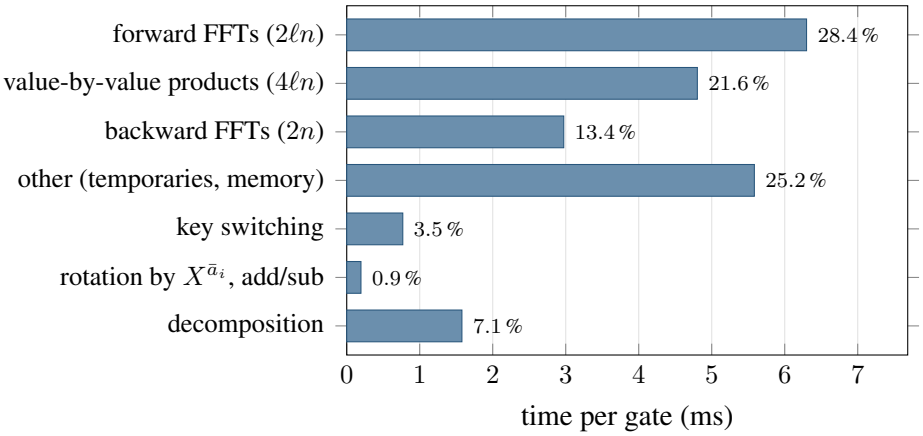


Figure 22. Where the time of a NAND gate goes. We measured each operation on its own and multiplied its time by the number of times it runs per gate; percentages are relative to the measured time of a gate. The value-by-value products are measured with the TRGSW rows read from main memory, as in the blind rotation. "Other" is the measured gate time minus all the listed parts.

5.4. Memory

Most of the memory goes to the two parts of the evaluation key. The bootstrapping key takes 59.1 MiB ($n \cdot 2\ell \cdot 2 \cdot \frac{N}{2}$ complex numbers of 16 bytes) and the key-switching key takes 78.9 MiB ($N \cdot t \cdot B_{\text{ks}}$ ciphertexts of $n+1$ torus elements of 4 bytes). An encrypted bit, on the other hand, takes only $(n+1) \cdot 4 = 2524$ bytes, which is still about $20{,}000$ times more than the bit itself. The peak memory of our benchmark program is 143 MiB, less than the 253 MiB of the original library, which also keeps the bootstrapping key as coefficients.

5.5. Noise

Bootstrapping resets the noise. First, a direct check of the property explained in Section 3.7. We encrypted $+\frac{1}{8}$ with a very large noise, more than 300 times the noise of a normal ciphertext, and bootstrapped each ciphertext once. Figure 23 shows the result: after the bootstrapping, the noise is the same as for any other input.

Bounds and average case. The bounds of Eqs. (5) and (6) are worst cases: they assume that every digit has the largest possible size β and that every key-switching digit

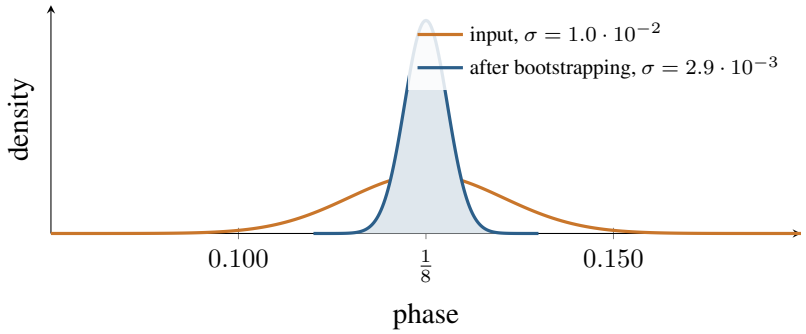


Figure 23. Bootstrapping resets the noise. Orange: the distribution of the phase of encryptions of $+\frac{1}{8}$ created with standard deviation $1.0 \cdot 10^{-2}$, more than 300 times the noise of a normal ciphertext. Blue: the phase after one bootstrapping of each of them, with standard deviation $2.9 \cdot 10^{-3}$, the same as for any other input.

is nonzero. We get a better prediction by noting that the digits are close to uniformly distributed. A uniform digit in $[-\beta, \beta)$ has $\mathbb{E}[d^2] \approx \beta^2/3$, so the average variance of the blind rotation is $n \cdot 2\ell N(\beta^2/3)\vartheta_{\mathrm{BK}} \approx 4.7 \cdot 10^{-6}$, a third of the bound.

Key switching needs more care, because the table is created only once. For a fixed key, each entry has a *fixed* noise value. Let X be the noise added at position (i, j), which is $e_{i,j,v}$ for a digit v uniform in $\{0, \ldots, B_{\mathrm{ks}} - 1\}$, with $e_{i,j,0} = 0$. Taking the average over the inputs with the table fixed, and then over the random table, gives

$$\mathbb{E}\big[\mathrm{Var}(X \mid \mathrm{KSK})\big] = \Big(\frac{B_{\mathrm{ks}} - 1}{B_{\mathrm{ks}}}\Big)^2 \vartheta_{\mathrm{KSK}}, \qquad \mathrm{Var}\big(\mathbb{E}[X \mid \mathrm{KSK}]\big) = \frac{B_{\mathrm{ks}} - 1}{B_{\mathrm{ks}}^2}\,\vartheta_{\mathrm{KSK}}. \quad (7)$$

Adding the first term over the tN positions predicts a variance of $4.3 \cdot 10^{-6}$ for key switching within one key, so the total variance of a gate output, for a fixed key, should be about $9.0 \cdot 10^{-6}$. The second term predicts that the average error for a given key is not zero: each key has its own small *bias*, with standard deviation $\sqrt{3tN/16}\,\sigma_{\mathrm{LWE}} \approx 1.2 \cdot 10^{-3}$ across keys.

Measurements. To test these predictions, we created 10 independent keys and bootstrapped 1,000 random NAND inputs with each of them (Table 8 and Figure 24). The measured variances are far below the worst-case bounds and within a few percent of the average-case predictions, both after blind rotation and after key switching. The standard deviation of the average error of each key, $9.8 \cdot 10^{-4}$, agrees with the predicted $1.2 \cdot 10^{-3}$ (with only ten keys, this estimate has an uncertainty of about 25%). The bias is harmless, since it is about a hundred times smaller than the margin, but it explains why the average error of the original library is smaller (Table 7): when it creates the key-switching table, the library draws all the noise values first and subtracts their average, which removes most of the bias.

6. Conclusion

We implemented TFHE gate bootstrapping from scratch, in about 526 lines of C++ without external libraries, and tested it layer by layer. With the parameters of the original

Table 8. Variance of the output error of the gate bootstrapping: worst-case bound, average-case prediction and measurement (10 keys $\times$ 1,000 gates; variance within each key, averaged over the keys).

	Bound	Average case	Measured
After blind rotation and extraction	$1.4 \cdot 10^{-5}$	$4.7 \cdot 10^{-6}$	$4.6 \cdot 10^{-6}$
After key switching (within a key)	$2.2 \cdot 10^{-5}$	$9.0 \cdot 10^{-6}$	$9.0 \cdot 10^{-6}$
Average error of a key, std. deviation	—	$1.2 \cdot 10^{-3}$	$9.8 \cdot 10^{-4}$

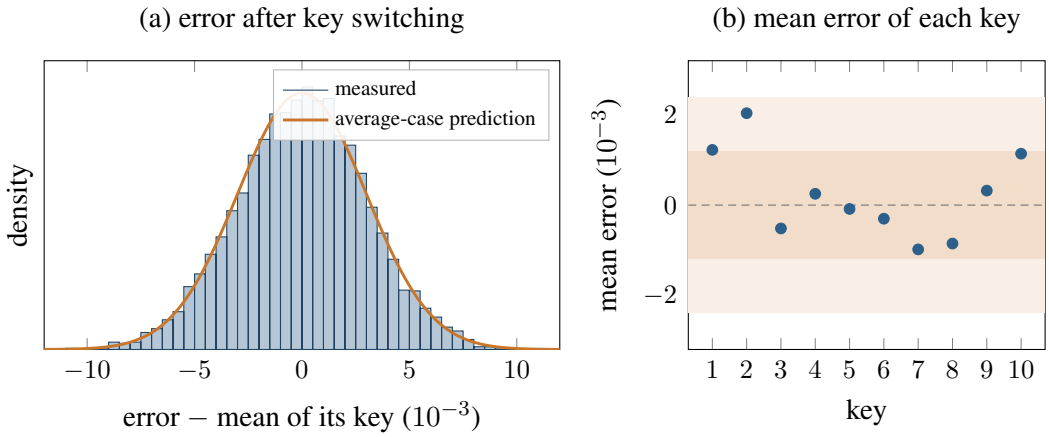


Figure 24. Output noise of the gate bootstrapping, for 10 keys and 1,000 gates per key. (a) Histogram of the error after key switching, measured from the average error of its own key, compared with a Gaussian with the predicted variance $9.0 \cdot 10^{-6}$**. (b) The average error of each key; the shaded bands show** ± 1 **and** ± 2 **predicted standard deviations of the bias of Eq.** (7)**. Both are very small compared with the margin** $\frac{1}{8} = 125 \cdot 10^{-3}$**.**

library, a gate takes 22.9 ms on one core. This is 2.3 times slower than the original library with its assembly FFT and 4.9 times faster than its portable version, and our implementation uses less memory. We saw no decryption error in 12,500 gates.

Building the scheme step by step made the role of each part clear. The external product keeps the noise growth additive, which allows hundreds of operations in one bootstrapping. The offset trick makes the signed decomposition almost free. Most of the time is spent in the FFT and in reading the bootstrapping key from memory, since the key is much larger than the processor caches. Our noise measurements agree with an average-case analysis, which also explained a small key-dependent bias in our key switching that the original library removes by centering its noise.

There are several ways to continue this work, roughly from the easiest to the hardest: centering the key-switching noise as the original library does; removing the temporary vectors from the inner loops; storing the bootstrapping key in one contiguous block and loading the next TRGSW ciphertext in advance; using AVX2/FMA instructions in the FFT; running independent gates in parallel threads; and using other test polynomials to compute more general functions [Chillotti et al. 2021].

References

[Chillotti et al. 2016a] Chillotti, I., Gama, N., Georgieva, M., and Izabachène, M. (2016a). Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. In *Advances in Cryptology – ASIACRYPT 2016*, volume 10031 of *LNCS*, pages 3–33. Springer.

[Chillotti et al. 2016b] Chillotti, I., Gama, N., Georgieva, M., and Izabachène, M. (2016b). TFHE: Fast fully homomorphic encryption library. `https://github.com/tfhe/tfhe`.

[Chillotti et al. 2020] Chillotti, I., Gama, N., Georgieva, M., and Izabachène, M. (2020). TFHE: Fast fully homomorphic encryption over the torus. *Journal of Cryptology*, 33(1):34–91.

[Chillotti et al. 2021] Chillotti, I., Joye, M., and Paillier, P. (2021). Programmable bootstrapping enables efficient homomorphic inference of deep neural networks. In *Cyber Security Cryptography and Machine Learning (CSCML 2021)*, volume 12716 of *LNCS*, pages 1–19. Springer.

[Cooley and Tukey 1965] Cooley, J. W. and Tukey, J. W. (1965). An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*, 19(90):297–301.

[Crandall and Fagin 1994] Crandall, R. and Fagin, B. (1994). Discrete weighted transforms and large-integer arithmetic. *Mathematics of Computation*, 62(205):305–324.

[Ducas and Micciancio 2015] Ducas, L. and Micciancio, D. (2015). FHEW: Bootstrapping homomorphic encryption in less than a second. In *Advances in Cryptology – EUROCRYPT 2015*, volume 9056 of *LNCS*, pages 617–640. Springer.

[Gentry 2009] Gentry, C. (2009). Fully homomorphic encryption using ideal lattices. In *Proceedings of the 41st Annual ACM Symposium on Theory of Computing (STOC)*, pages 169–178.

[Gentry et al. 2013] Gentry, C., Sahai, A., and Waters, B. (2013). Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In *Advances in Cryptology – CRYPTO 2013*, volume 8042 of *LNCS*, pages 75–92. Springer.

[Joye 2022] Joye, M. (2022). Guide to fully homomorphic encryption over the [discretized] torus. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2022(4):661–692.

[Lyubashevsky et al. 2010] Lyubashevsky, V., Peikert, C., and Regev, O. (2010). On ideal lattices and learning with errors over rings. In *Advances in Cryptology – EUROCRYPT 2010*, volume 6110 of *LNCS*, pages 1–23. Springer.

[Regev 2005] Regev, O. (2005). On lattices, learning with errors, random linear codes, and cryptography. In *Proceedings of the 37th Annual ACM Symposium on Theory of Computing (STOC)*, pages 84–93.

[Rivest et al. 1978] Rivest, R. L., Adleman, L., and Dertouzos, M. L. (1978). On data banks and privacy homomorphisms. In *Foundations of Secure Computation*, pages 169–180. Academic Press.